M.TECH · DATABASE MANAGEMENT SYSTEMS

Relational Algebra & Calculus — Visual Study Book

Relations and keys → algebra → joins, division & grouping → tuple calculus → domain calculus & Codd's theorem. Every query written four ways — algebra, TRC, DRC and SQL — with the result computed on a real database.

Unit I · Relational Model | Unit II · Algebra I | Unit III · Algebra II | Unit IV · Tuple Calculus | Unit V · Domain Calculus & Workbook

41 original diagrams · every result table computed by running the SQL · 12-query master workbook · practice problems with answers

Table of Contents

How to use this book ... 3

The Relational Model & Query Languages ... 5

1.1 Why do we need query languages? ... 5
1.2 The relational model — relation, tuple, attribute ... 6
1.3 Keys — how a row is identified ... 7
1.4 Integrity constraints ... 8
1.5 Our sample database ... 9
1.6 Set semantics and the closure property ... 9
1.7 How a query is processed ... 10

Relational Algebra I — The Fundamental Operations ... 12

2.1 The basic operators ... 12
2.2 Selection (σ) — choosing rows ... 12
2.3 Projection (π) — choosing columns ... 14
2.4 Composing operators ... 15
2.5 Rename (ρ) ... 16
2.6 Set operations — union, intersection, difference ... 17
2.7 Cartesian product (×) ... 20
2.8 From product to join ... 21

Relational Algebra II — Joins, Division, Grouping & Optimization ... 23

3.1 Join (⋈) — combining related relations ... 23
3.2 Joining three relations ... 25
3.3 Outer joins ... 26
3.4 Semi-join (⋉) and anti-join (▷) ... 28
3.5 Writing "not", "both" and "either" queries ... 29
3.6 Self-join ... 31
3.7 Division (÷) — the "for ALL" operator ... 32
3.8 Grouping and aggregation (γ) ... 34
3.9 Extended operators and NULLs ... 36
3.10 Equivalent expressions and optimization ... 36
3.11 Algebra ⇄ SQL translation table ... 38

Tuple Relational Calculus (TRC) ... 39

4.1 From algebra to calculus ... 39
4.2 TRC syntax ... 40
4.3 Free and bound variables ... 41
4.4 Selection and projection ... 41
4.5 Joins with ∃ ... 42
4.6 The two quantifiers ... 43
4.7 "Not" queries with ¬∃ ... 45
4.8 "For all" queries with ∀ (the division pattern) ... 46
4.9 Safety of expressions ... 46
4.10 Union, intersection and difference in TRC ... 47
4.11 Algebra ⇄ TRC dictionary ... 48

Domain Calculus, Codd's Theorem & the Query-Writing Workbook ... 50

5.1 Domain relational calculus (DRC) ... 50
5.2 TRC versus DRC ... 52
5.3 Codd's theorem — algebra equals calculus ... 52
5.4 What algebra and calculus cannot express ... 54
5.5 Query-By-Example (QBE) in brief ... 54
5.6 The master workbook — one question, four languages ... 54
5.7 A method for writing any query ... 61
5.8 Common mistakes and how to avoid them ... 62
5.9 Practice problems with solutions ... 62
5.10 Quick viva answers ... 66

Quick comparison sheet ... 67

Exam-day revision checklist ... 69

Recommended resources ... 70

How to use this book

The 3-step exam answer formula

⭐ **Exam tip — Write every answer in this order**

1. **Definition** — one line (the blue 📘 boxes give you this ready-made).
2. **Notation and explanation** — the symbol, the meaning in 3–4 lines, keywords in **bold**.
3. **Diagram and example** — draw the figure; for query questions show the **expression** and the **result table**.

What the coloured blocks mean

Block	What it gives you
RELATIONAL ALGEBRA $\sigma_{age>20}(Student)$	An algebra expression (violet)
TUPLE / DOMAIN CALCULUS { t | Student(t) }	A TRC or DRC query (cyan)
Dark code block + result table	The SQL equivalent, **run on the sample database**; the table under it is the real output
📘 Definition · 🏭 Real-life use · 💡 Why it matters	Opening line, application, and the reason a concept exists
⭐ Exam tip · 🧠 Yaad rakho · ⚠️ Careful	Marks-scoring points, a Hinglish memory hook, common traps
❓ Likely exam questions	Past-paper style questions at the end of every unit

The book follows the standard university syllabus for relational algebra and relational calculus. No lecture notes were supplied, so check any university-specific notation (for example the symbol for outer joins) against your own slides.

Syllabus map

Unit	Focus	Main figures to practise
I	Relations, tuples, keys, integrity, sample database, set semantics	Key hierarchy · FK links · query pipeline
II	σ, π, ρ, ∪ ∩ −, ×, product → join	σ/π pictures · Venn sets · product grid
III	Joins, outer joins, semi/anti-join, self-join, division, grouping, optimization	Join family · division · push-selection-down
IV	TRC: syntax, free/bound variables, ∃ ∀, "not" and "all", safety	TRC anatomy · quantifiers · ∀ pattern
V	DRC, Codd's theorem, power, QBE, 12-query workbook, practice	Codd triangle · query-writing method

The sample database

Three small relations are used throughout: Student(sid, sname, age, dept), Course(cid, title, credits, dept) and Enrolled(sid, cid, grade). The data is listed in Unit I, §1.5 — keep that page open while you read the examples.

🏭 Real-life use case — Where this is used

Every SQL database turns your query into relational algebra internally, and its optimizer rewrites that algebra to run faster. Calculus is the logic behind SQL's `EXISTS` and `NOT EXISTS`. Knowing both lets you **write correct queries first time** and **read query plans** confidently.

Practice reading: GeeksforGeeks — Relational Algebra · Tuple Relational Calculus · Domain Relational Calculus

UNIT I

The Relational Model & Query Languages

Why query languages | Relations & tuples | Keys | Integrity | Sample database | Set semantics | How queries run

1.1 Why do we need query languages?

📘 **Definition**

A **query language** is a formal language for asking questions of a database and for changing its data. For relational databases there are two **formal** families — **relational algebra** (procedural: *how* to get the answer) and **relational calculus** (declarative: *what* the answer looks like) — and one **practical** language, **SQL**, built on both.

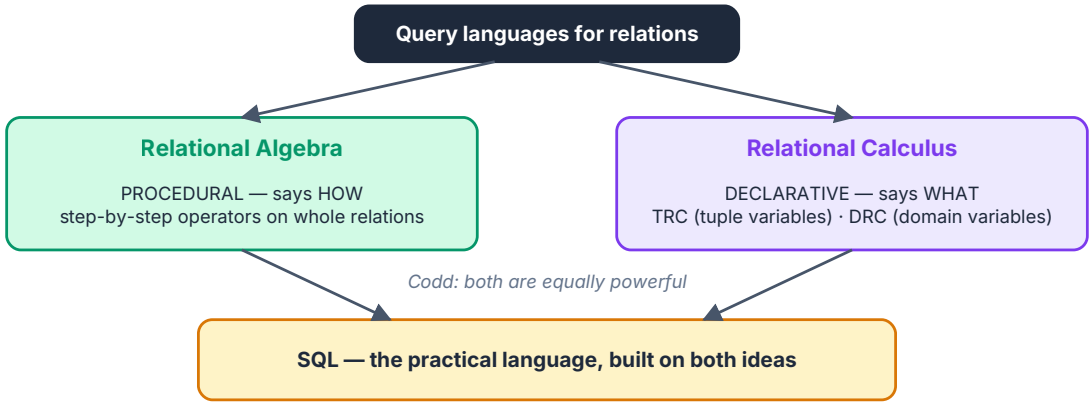


Fig. 1 — Relational algebra and calculus are the theory; SQL is what you type

	Relational algebra	Relational calculus	SQL
Style	Procedural	Declarative (non-procedural)	Mostly declarative
You write	A chain of operators	A logical formula describing the result	SELECT ... FROM ... WHERE
Variables	None — works on whole relations	Tuple variables (TRC) or domain variables (DRC)	Tuple variables (aliases)
Used for	Internal plans, optimization, theory	Theory, query semantics, QBE	Real databases

🏭 Real-life use case — Recipe vs restaurant order

Algebra is a recipe: *"take the student table, keep only CSE rows, keep only the name column."* **Calculus** is ordering at a restaurant: *"I want the names of all CSE students"* — you do not say how the kitchen should do it. The kitchen (the query optimizer) decides.

🧠 Yaad rakho

Algebra = **kaise** (how, step by step). Calculus = **kya** (what, sirf result batao). Dono ek hi power rakhte hain — Codd ne prove kiya (Unit V).

📖 GFG: Relational Algebra in DBMS · Relational Calculus in DBMS

1.2 The relational model — relation, tuple, attribute

📘 Definition

A **relation** is a **set of tuples** that all have the same attributes. In everyday words: a **table** whose rows are tuples and whose columns are attributes. Each attribute draws its values from a **domain** (the set of allowed values).

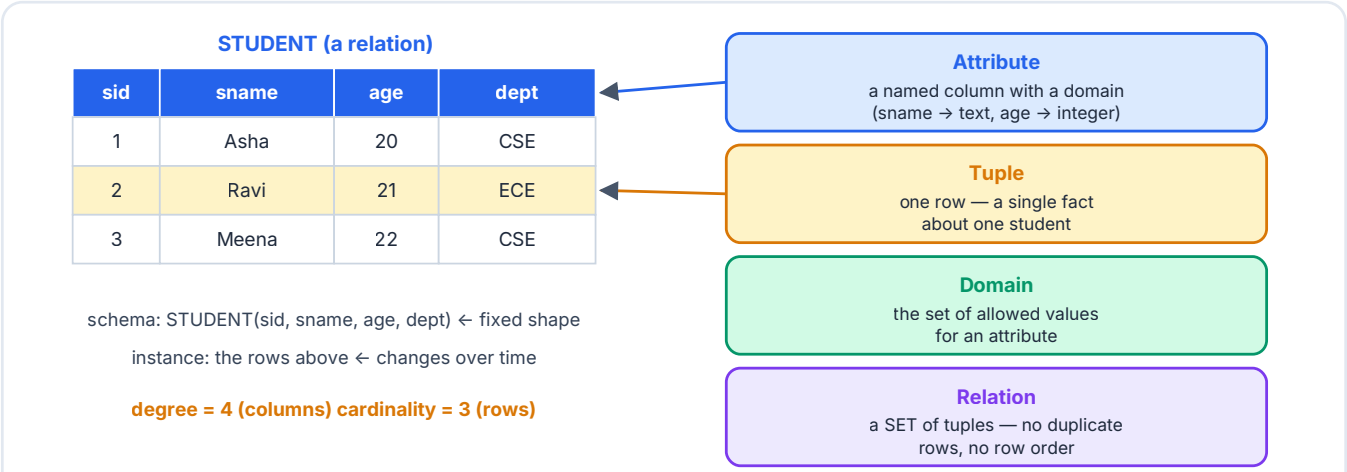


Fig. 2 — One relation with its schema, instance, degree and cardinality

Formal term	Everyday term	Meaning
Relation	Table	A set of tuples sharing one schema
Tuple	Row / record	One fact, e.g. one student
Attribute	Column / field	A named property with a domain
Domain	Data type + rules	Allowed values, e.g. age: integers 16–60
Degree (arity)	Number of columns	STUDENT has degree 4
Cardinality	Number of rows	Changes as data changes
Schema	Table definition	STUDENT(sid, sname, age, dept) — fixed
Instance	Current contents	The rows right now — changes over time

⚠️ **Careful — Four properties every relation has**

- **No duplicate tuples** — it is a set.
- **Tuple order does not matter** — there is no "first row" in theory.
- **Attribute order does not matter** — columns are identified by name.
- **Atomic values** — each cell holds one value, not a list (first normal form).

⭐ **Exam tip**

"Differentiate between schema and instance" and "degree vs cardinality" are favourite 2-mark questions. Remember: **schema = shape (fixed)**, **instance = data (changes)**; **degree = columns**, **cardinality = rows**.

1.3 Keys — how a row is identified

📘 **Definition**

A **superkey** is any set of attributes that uniquely identifies a tuple. A **candidate key** is a *minimal* superkey. The **primary key** is the candidate key the designer chooses; the rest are **alternate keys**. A **foreign key** is an attribute (set) in one relation that refers to the primary key of another.

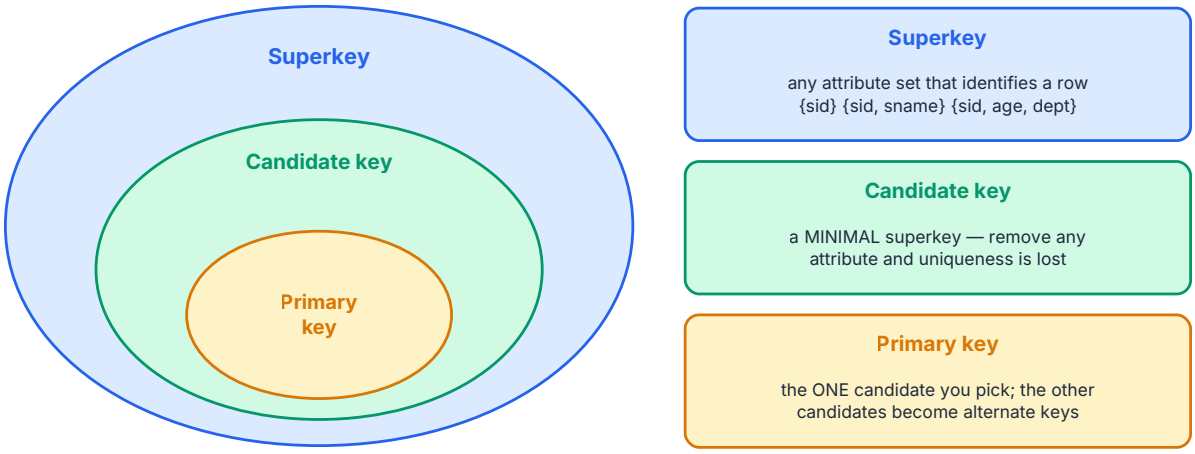


Fig. 3 — Every primary key is a candidate key, and every candidate key is a superkey

Key type	Example in our database
Superkey	{sid}, {sid, sname}, {sid, sname, age, dept} — anything containing sid
Candidate key	{sid} for STUDENT; {sid, cid} for ENROLLED (neither column alone is unique)
Primary key	sid in STUDENT, cid in COURSE, (sid, cid) in ENROLLED
Composite key	(sid, cid) in ENROLLED — a key made of more than one attribute
Foreign key	ENROLLED.sid → STUDENT.sid and ENROLLED.cid → COURSE.cid

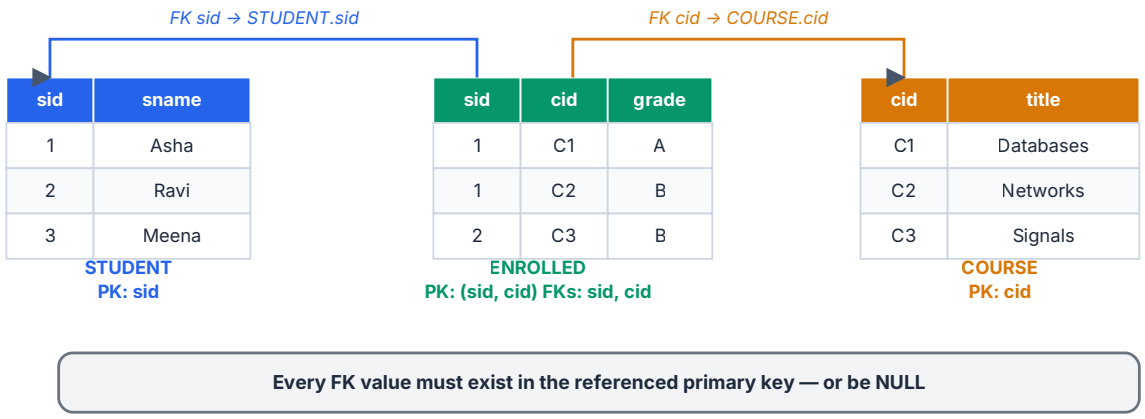


Fig. 4 — Foreign keys link ENROLLED to STUDENT and COURSE — these links are what joins follow

💡 **Why it matters — Why keys matter for queries**

Almost every join you write is **foreign key = primary key**. When you are unsure how to connect two tables, look for the FK arrow between them.

🧠 **Yaad rakho**

Superkey ⊃ Candidate ⊃ Primary. **Candidate = minimal** — ek bhi attribute hatao to unique nahi rahega.

1.4 Integrity constraints

📘 **Definition**

Integrity constraints are rules the database enforces so that stored data stays correct and consistent.

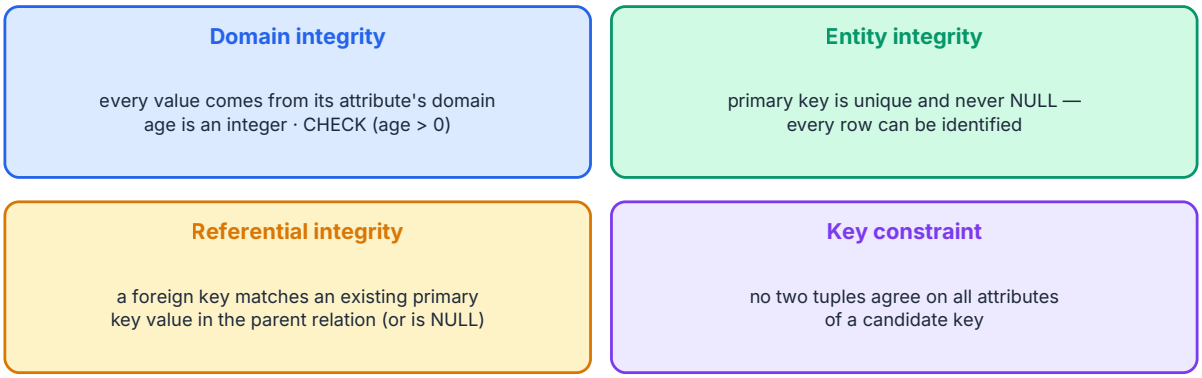


Fig. 5 — The four constraints every relational design relies on

⚠️ **Careful — NULL in keys**

A primary key can **never** be NULL (entity integrity). A foreign key **may** be NULL — meaning "refers to nothing" — but if it has a value, that value must exist in the parent table (referential integrity).

1.5 Our sample database

Every example in this book uses the same three small relations, so you can follow every answer by eye. **Every result table in the book was produced by actually running the query** on this data — nothing is typed by hand.

STUDENT

sid	sname	age	dept
1	Asha	20	CSE
2	Ravi	21	ECE
3	Meena	22	CSE
4	Karan	20	ME
5	Sara	21	CSE
6	Dev	23	ECE

COURSE

cid	title	credits	dept
C1	Databases	4	CSE
C2	Networks	3	CSE
C3	Signals	4	ECE
C4	Thermo	3	ME
C5	AI	4	CSE

ENROLLED

sid	cid	grade
1	C1	A
1	C2	B
1	C5	A
2	C3	B
2	C1	C
3	C1	A
3	C2	A
3	C5	B
4	C4	B
5	C1	B
5	C2	A

- **Dev** (sid 6) has no enrolment — useful for outer joins, anti-joins and "not" queries.
- Sid 2 (Ravi, ECE) takes a CSE course (C1) — useful for "other department" queries.
- Only **Asha and Meena** take all three CSE courses (C1, C2, C5) — the answer to the division query in Unit III.
- **dept** appears in both STUDENT and COURSE — a deliberate trap for natural joins (Unit III).

⭐ **Exam tip — Run it yourself**

The script below works in SQLite, MySQL, PostgreSQL and Oracle (Oracle: use `NUMBER` / `VARCHAR2` and write each INSERT separately). Practising each query on real data is the fastest way to learn.

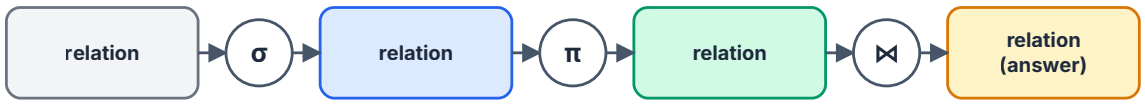
```sql
CREATE TABLE Student (sid INT PRIMARY KEY, sname VARCHAR(20), age INT, dept VARCHAR(5));
CREATE TABLE Course  (cid VARCHAR(3) PRIMARY KEY, title VARCHAR(20), credits INT, dept VARCHAR(5));
CREATE TABLE Enrolled(sid INT REFERENCES Student(sid), cid VARCHAR(3) REFERENCES Course(cid),
                      grade CHAR(1), PRIMARY KEY (sid, cid));

INSERT INTO Student VALUES (1,'Asha',20,'CSE'), (2,'Ravi',21,'ECE'), (3,'Meena',22,'CSE'),
                           (4,'Karan',20,'ME'),  (5,'Sara',21,'CSE'), (6,'Dev',23,'ECE');
INSERT INTO Course  VALUES ('C1','Databases',4,'CSE'), ('C2','Networks',3,'CSE'), ('C3','Signals',
4,'ECE'),
                           ('C4','Thermo',3,'ME'),     ('C5','AI',4,'CSE');
INSERT INTO Enrolled VALUES (1,'C1','A'),(1,'C2','B'),(1,'C5','A'),(2,'C3','B'),(2,'C1','C'),
                            (3,'C1','A'),(3,'C2','A'),(3,'C5','B'),(4,'C4','B'),(5,'C1','B'),
(5,'C2','A');
```

1.6 Set semantics and the closure property

📘 **Definition**

Closure property: every relational-algebra operator takes relation(s) as input and produces a **relation** as output — so the output of one operator can be the input of the next, to any depth.



Input = relation(s) → operator → output = a relation again

This closure property lets you nest operators to any depth

Fig. 6 — Closure lets operators be chained into long expressions

⚠ Careful — Algebra uses SETS — SQL uses BAGS

In relational algebra a relation is a **set**, so duplicate rows can never exist. Plain SQL keeps duplicates unless you write `DISTINCT`. Compare the same projection of **dept**:

```sql
SELECT dept FROM Student;
```

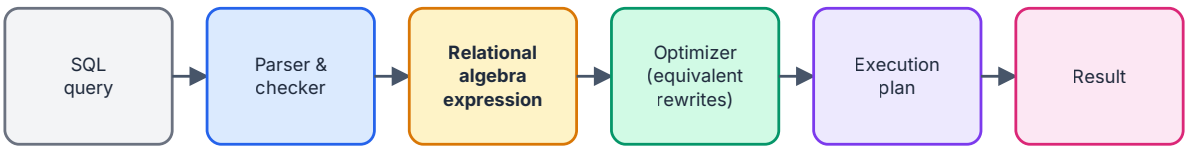
RESULT — 6 ROWS

dept
CSE
ECE
CSE
ME
CSE
ECE

Six rows in SQL (a *bag*), but algebra's $\pi_{dept}(Student)$ gives only **CSE, ECE, ME** — three rows. Add `SELECT DISTINCT` when you translate algebra to SQL.

1.7 How a query is processed

What a database does with the SQL you type



Relational algebra is the internal language of every SQL engine

The optimizer swaps an expression for a cheaper equivalent one

Fig. 7 — Relational algebra sits in the middle of every SQL engine

When you submit SQL, the database parses it, converts it to a **relational-algebra expression**, the **optimizer** rewrites it into a cheaper but equivalent expression (Unit III, §3.10), and an execution plan finally runs. Learning algebra therefore teaches you what the engine really does.

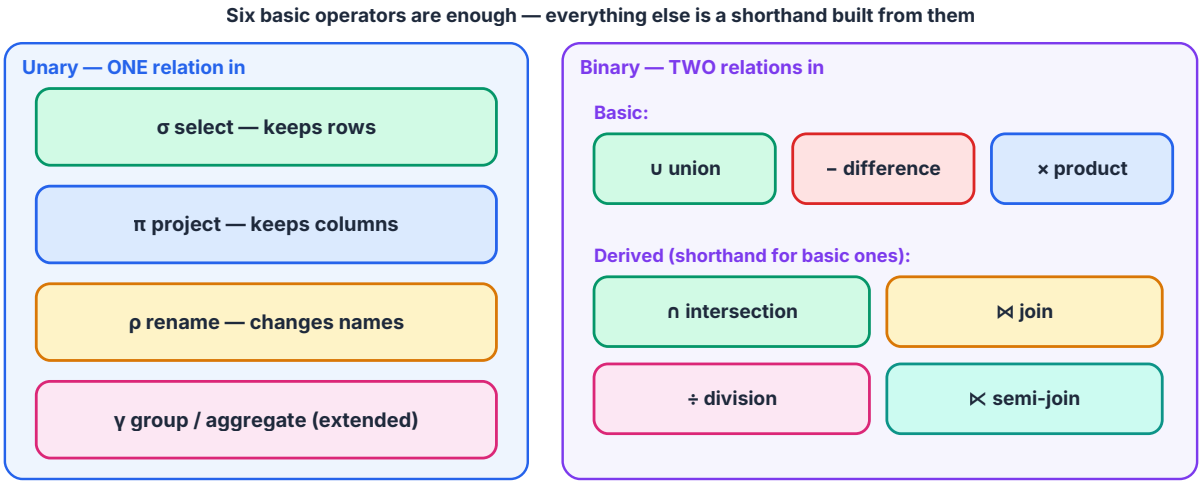


Fig. 8 — The operators of relational algebra, grouped by how many relations they read

Algebra	Name	SQL counterpart
σ	Selection	`WHERE`
π	Projection	`SELECT DISTINCT col, …`
ρ	Rename	`AS` (aliases)
∪ ∩ −	Union, intersection, difference	`UNION`, `INTERSECT`, `EXCEPT` (Oracle: `MINUS`)
×	Cartesian product	`CROSS JOIN` or a comma in `FROM`
⋈	Join	`JOIN … ON`, `NATURAL JOIN`
÷	Division	double `NOT EXISTS` or `GROUP BY … HAVING COUNT`
γ	Grouping / aggregation	`GROUP BY`, `SUM`, `COUNT`, …

❓ Likely exam questions — Unit I

- Define relation, tuple, attribute, domain, degree and cardinality.
- Differentiate between relational algebra and relational calculus.
- Explain superkey, candidate key, primary key and foreign key with examples.
- Explain the four integrity constraints.
- Why is relational algebra said to be "closed"?
- How is a relation different from a SQL table?
- Show where relational algebra fits in query processing.

UNIT II

Relational Algebra I — The Fundamental Operations

Selection σ | Projection π | Composition | Rename ρ | Union ∪ ∩ − | Product × | Product → Join

2.1 The basic operators

📘 **Definition**

Relational algebra is a procedural query language: a set of operators, each taking one or two relations and returning a relation. A query is an **expression** built by nesting operators.

Operator	Symbol	Inputs	What it does	SQL
Selection	σ	1	Keeps the **rows** that satisfy a condition	`WHERE`
Projection	π	1	Keeps chosen **columns**, removes duplicates	`SELECT DISTINCT`
Rename	ρ	1	Renames a relation and/or its attributes	`AS`
Union	∪	2	Rows in either relation	`UNION`
Difference	−	2	Rows in the first but not the second	`EXCEPT`
Product	×	2	Every row paired with every row	`CROSS JOIN`

⭐ **Exam tip — Notation used in this book**

Relation names start with a capital (Student), attributes are lower-case (sname). Subscripts hold conditions or column lists: $\sigma_{age>20}(\text{Student})$. An arrow ← names an intermediate result: $R1 \leftarrow \pi_{sid}(\text{Enrolled})$.

2.2 Selection (σ) — choosing rows

📘 **Definition**

Selection $\sigma_{\theta}(R)$ returns all tuples of R for which the condition **θ** is TRUE. The result has the **same attributes** as R and at most as many tuples.

σ age > 20 (Student) — keeps ROWS, every column stays

sid	sname	age	dept
1	Asha	20	CSE
2	Ravi	21	ECE
3	Meena	22	CSE
4	Karan	20	ME
5	Sara	21	CSE
6	Dev	23	ECE

σ age > 20 →

sid	sname	age	dept
2	Ravi	21	ECE
3	Meena	22	CSE
5	Sara	21	CSE
6	Dev	23	ECE

4 of 6 rows survive

green = predicate TRUE (kept) gray = FALSE (dropped)

Fig. 9 — Selection filters rows; every column is kept

The condition can compare an attribute with a constant or with another attribute, using = ≠ < ≤ > ≥, and can be combined with ∧ (and), ∨ (or) and ¬ (not).

Example 1 — a simple condition

Question: students older than 20.

RELATIONAL ALGEBRA

$$\sigma_{age>20}(Student)$$

```sql
SELECT * FROM Student WHERE age > 20;
```

RESULT — 4 ROWS

sid	sname	age	dept
2	Ravi	21	ECE
3	Meena	22	CSE
5	Sara	21	CSE
6	Dev	23	ECE

Example 2 — two conditions with ∧

Question: CSE students aged 21 or more.

RELATIONAL ALGEBRA

$$\sigma_{dept='CSE' \wedge age\geq 21}(Student)$$

```sql
SELECT * FROM Student WHERE dept = 'CSE' AND age >= 21;
```

RESULT — 2 ROWS

sid	sname	age	dept
3	Meena	22	CSE
5	Sara	21	CSE

Example 3 — negation

Question: students who are not in CSE.

RELATIONAL ALGEBRA

$$\sigma_{\neg(dept='CSE')}(Student) \equiv \sigma_{dept \neq 'CSE'}(Student)$$

```sql
SELECT * FROM Student WHERE NOT (dept = 'CSE');
```

RESULT — 3 ROWS

sid	sname	age	dept
2	Ravi	21	ECE
4	Karan	20	ME
6	Dev	23	ECE

⚠️ **Careful — Selection keeps only TRUE**

A row is kept only when the condition is **TRUE**. If the condition is unknown (because of a NULL), the row is dropped. Never write `= NULL` — use `IS NULL` in SQL.

🧠 **Yaad rakho**

σ = **Sigma = Select rows** (S for Select, S for Sigma). Rows kam hoti hain, columns wahi rehte hain.

2.3 Projection (π) — choosing columns

📘 **Definition**

Projection $\pi_{A1,\ldots,Ak}(R)$ returns only the listed attributes of R, in that order, and then **removes duplicate tuples**.

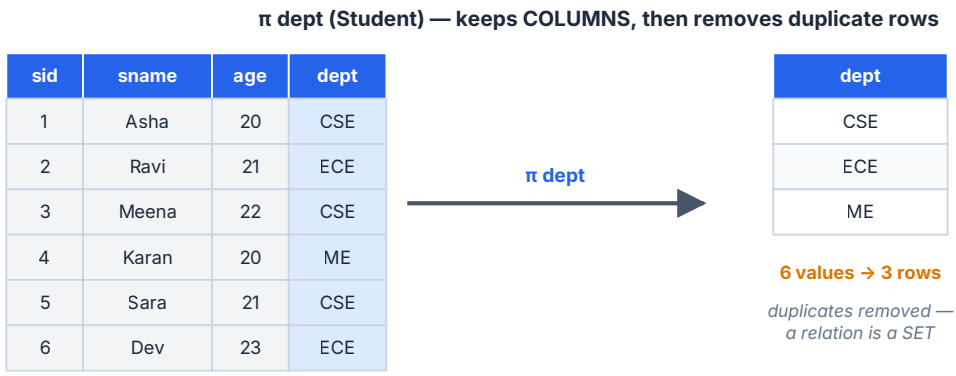


Fig. 10 — Projection keeps columns, then collapses duplicate rows

RELATIONAL ALGEBRA

$\pi_{dept}(Student)$

```sql
SELECT DISTINCT dept FROM Student;
```

RESULT — 3 ROWS

dept
CSE
ECE
ME

Six students but only three departments: after dropping the other columns, identical rows collapse into one. Projecting a **key** never loses rows; projecting a non-key can.

RELATIONAL ALGEBRA

$\pi_{sname,\ dept}(Student)$

```sql
SELECT DISTINCT sname, dept FROM Student;
```

RESULT — 6 ROWS

sname	dept
Asha	CSE
Ravi	ECE
Meena	CSE
Karan	ME
Sara	CSE
Dev	ECE

⚠️ **Careful — SQL needs DISTINCT**

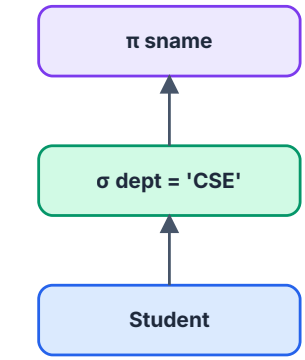
Algebra removes duplicates automatically; SQL does not. Translate π as `SELECT DISTINCT …` if the result might contain duplicates.

🧠 **Yaad rakho**

π = **Pi = Pick columns**. Rows kam ho sakti hain kyunki duplicates hat jaate hain.

2.4 Composing operators

Because the output is a relation, operators nest. Work from the **innermost** parentheses outwards.



data flows UP the tree (read bottom-up)

Read it as a story

1. take every row of Student
2. σ keeps only the CSE rows
3. π keeps only the sname column

Same thing in SQL

```
SELECT DISTINCT sname
FROM Student
WHERE dept = 'CSE';
```

Fig. 11 — The expression as a tree: data flows from the relation up to the answer

Question: names of CSE students.

RELATIONAL ALGEBRA

$\pi_{sname}(\sigma_{dept='CSE'}(Student))$

```sql
SELECT DISTINCT sname FROM Student WHERE dept = 'CSE';
```

RESULT — 3 ROWS

sname
Asha
Meena
Sara

⚠️ **Careful — Order matters**

$\sigma_{dept='CSE'}(\pi_{sname}(Student))$ is **invalid**: after π only *sname* remains, so there is no *dept* left to test. Rule of thumb: **filter first (σ), then pick columns (π)**.

2.5 Rename (ρ)

📘 **Definition**

Rename gives a relation (and optionally its attributes) a new name: $\rho_{S(id,\ name)}(R)$, or just $\rho_S(R)$ to rename the relation only, or $\rho_{a \to b}(R)$ to rename one attribute.

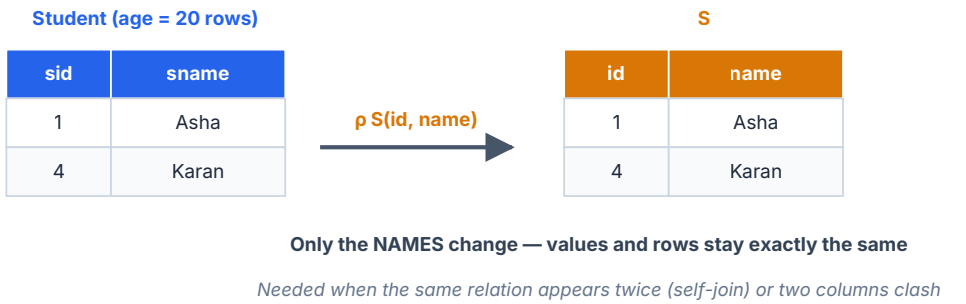


Fig. 12 — Only the names change; the data is untouched

RELATIONAL ALGEBRA

$$\rho_{S(id,\ name)}(\ \pi_{sid,\ sname}(\ \sigma_{age=20}(Student)\ )\ )$$

```sql
SQL
SELECT sid AS id, sname AS name FROM Student WHERE age = 20;
```

RESULT — 2 ROWS

id	name
1	Asha
4	Karan

💡 **Why it matters — When rename is essential**

- **Self-joins** — to compare a relation with itself you need two different names for it (Unit III).
- **Clashing attribute names** — after a product, both relations may have a column called *sid*.
- **Naming intermediate results** so long expressions stay readable.

2.6 Set operations — union, intersection, difference

📘 **Definition**

For two relations R and S: **R ∪ S** = tuples in R or S (or both); **R ∩ S** = tuples in both; **R − S** = tuples in R that are **not** in S. The two relations must be **union-compatible**.

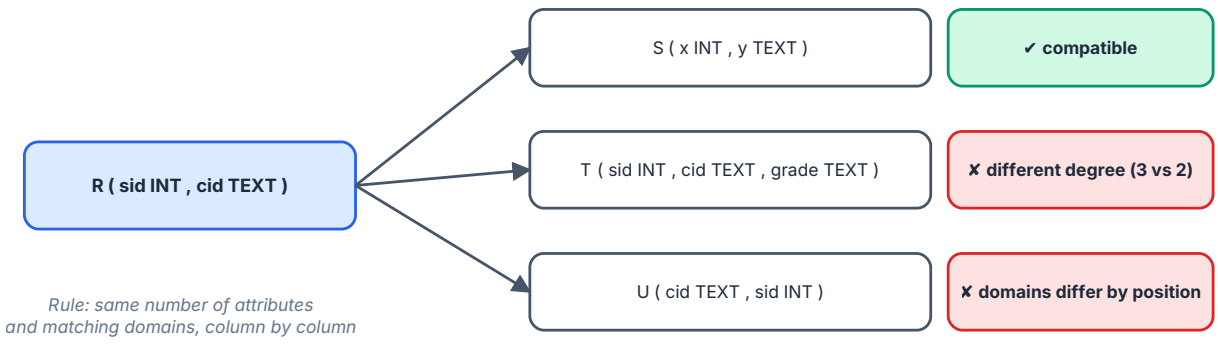


Fig. 13 — Union-compatible = same degree and matching domains, column by column

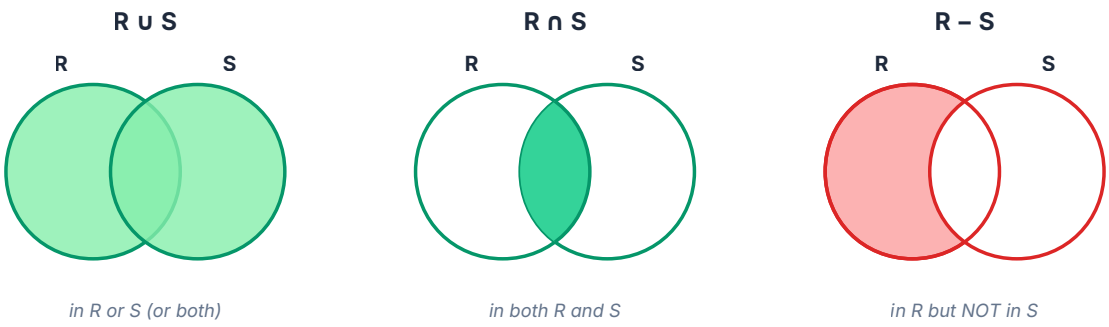


Fig. 14 — The three set operations as Venn diagrams

We use two small relations: the students enrolled in C1 and the students enrolled in C2.

RELATIONAL ALGEBRA

$$R1 \leftarrow \pi_{sid}(\sigma_{cid='C1'}(Enrolled)) \; R2 \leftarrow \pi_{sid}(\sigma_{cid='C2'}(Enrolled))$$

```sql
SELECT sid FROM Enrolled WHERE cid = 'C1';
```

RESULT — 4 ROWS

sid
1
2
3
5

```sql
SELECT sid FROM Enrolled WHERE cid = 'C2';
```

RESULT — 3 ROWS

sid
1
3
5

Union — "C1 or C2"

RELATIONAL ALGEBRA

$R1 \cup R2$

```sql
SELECT sid FROM Enrolled WHERE cid = 'C1'
UNION
SELECT sid FROM Enrolled WHERE cid = 'C2';
```

RESULT — 4 ROWS

sid
1
2
3
5

Intersection — "both C1 and C2"

RELATIONAL ALGEBRA

$R1 \cap R2$

```sql
SELECT sid FROM Enrolled WHERE cid = 'C1'
INTERSECT
SELECT sid FROM Enrolled WHERE cid = 'C2';
```

RESULT — 3 ROWS

sid
1
3
5

Difference — "C1 but not C2"

RELATIONAL ALGEBRA

$R1 - R2$

```sql
SELECT sid FROM Enrolled WHERE cid = 'C1'
EXCEPT
SELECT sid FROM Enrolled WHERE cid = 'C2';
```

RESULT — 1 ROW

sid
2

⭐ **Exam tip — Derived, commutative, associative**

Intersection is derived: R ∩ S = R − (R − S). Union and intersection are **commutative** and associative; difference is **not** — R1 − R2 ≠ R2 − R1 (here R2 − R1 is empty).

🧠 **Yaad rakho**

∪ = **ya (or)**, ∩ = **aur (and)**, − = **par nahi (but not)**. Union ke liye columns ka number aur type match hona zaroori hai.

2.7 Cartesian product (×)

📘 **Definition**

Cartesian product R × S pairs **every** tuple of R with **every** tuple of S. If R has m tuples and S has n, the result has **m × n** tuples and the attributes of both.

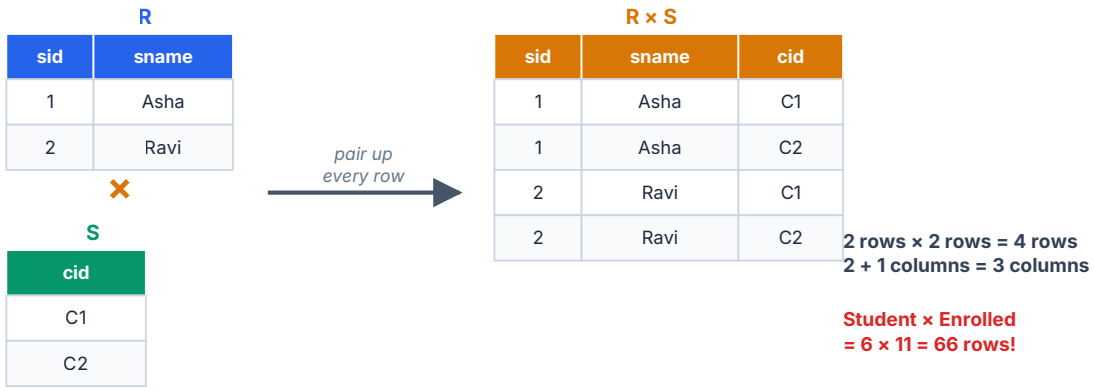


Fig. 15 — Two rows times two rows gives four rows

RELATIONAL ALGEBRA

$$\pi_{sid,sname}(\sigma_{sid\leq 2}(\text{Student})) \times \pi_{cid}(\sigma_{cid='C1' \vee cid='C2'}(\text{Course}))$$

```sql
SELECT * FROM (SELECT sid, sname FROM Student WHERE sid <= 2) a
CROSS JOIN (SELECT cid FROM Course WHERE cid IN ('C1','C2')) b;
```

RESULT — 4 ROWS

sid	sname	cid
1	Asha	C1
1	Asha	C2
2	Ravi	C1
2	Ravi	C2

The product of the whole Student × Enrolled is much bigger:

```sql
SELECT COUNT(*) AS rows_in_product FROM Student, Enrolled;
```

RESULT — 1 ROW

rows_in_product
66

> ⚠️ **Careful — Name clashes**
>
> After Student × Enrolled there are two attributes called **sid**. Qualify them as Student.sid and Enrolled.sid, or rename one with ρ.

2.8 From product to join



A product alone is mostly junk pairs — σ keeps only the related ones

× followed by σ on matching keys is exactly what we call a JOIN

Fig. 16 — A product followed by a selection on matching keys is a join

Nearly every product is followed by a selection that keeps only the **related** pairs. Here only the pairs where the student id matches survive:

RELATIONAL ALGEBRA

$$\pi_{\text{Student.sid, sname, cid, grade}}(\sigma_{\text{Student.sid = Enrolled.sid}}(\text{Student} \times \text{Enrolled}))$$

```sql
SELECT Student.sid, sname, cid, grade
FROM Student, Enrolled
WHERE Student.sid = Enrolled.sid;
```

RESULT — 11 ROWS

sid	sname	cid	grade
1	Asha	C1	A
1	Asha	C2	B
1	Asha	C5	A
2	Ravi	C3	B
2	Ravi	C1	C
3	Meena	C1	A
3	Meena	C2	A
3	Meena	C5	B
4	Karan	C4	B
5	Sara	C1	B
5	Sara	C2	A

This pattern is so common that algebra gives it its own operator — the **join** — covered in the next unit.

❓ Likely exam questions — Unit II

- Explain σ and π with examples; why does π remove duplicates?
- What does union-compatible mean? List the conditions.
- Explain ∪, ∩ and − with a Venn diagram and example.
- Show that R ∩ S = R − (R − S).
- What is the Cartesian product? How many tuples does it produce?
- Why is ρ needed? Give a case where it is essential.
- Why is $\sigma_{\text{dept='CSE'}}(\pi_{\text{sname}}(\text{Student}))$ invalid?
- List the six basic operators of relational algebra.

UNIT III

Relational Algebra II — Joins, Division, Grouping & Optimization

Theta / equi / natural join | Outer joins | Semi & anti-join | Self-join | Division ÷ | Aggregation γ | Optimization

3.1 Join (⋈) — combining related relations

Definition

A **theta join** $R \bowtie_\theta S$ is a Cartesian product followed by a selection: $R \bowtie_\theta S = \sigma_\theta(R \times S)$. An **equi-join** is a theta join whose condition uses only **=**. A **natural join** $R \bowtie S$ is an equi-join on **all attributes with the same name**, keeping each such column only once.

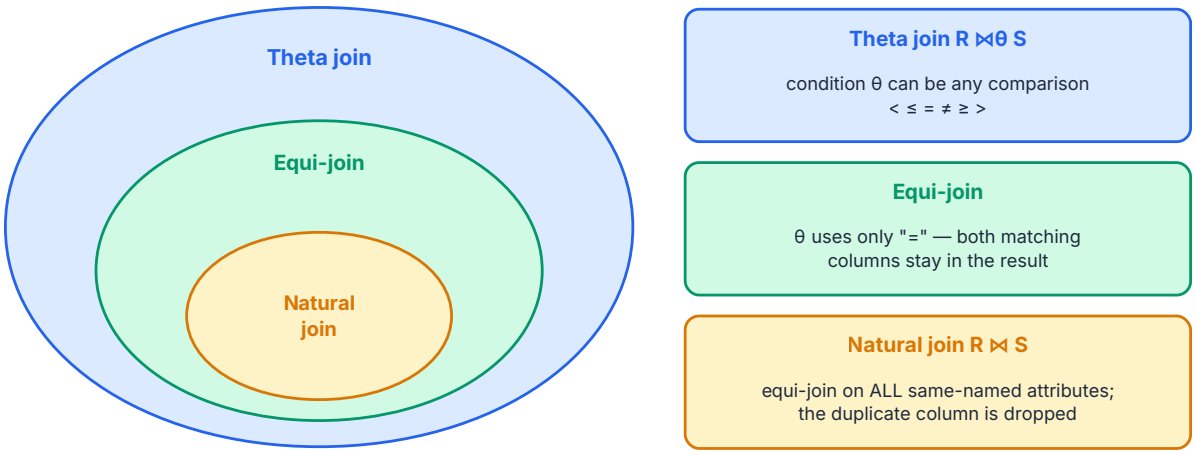


Fig. 17 — Natural join is a special equi-join, which is a special theta join

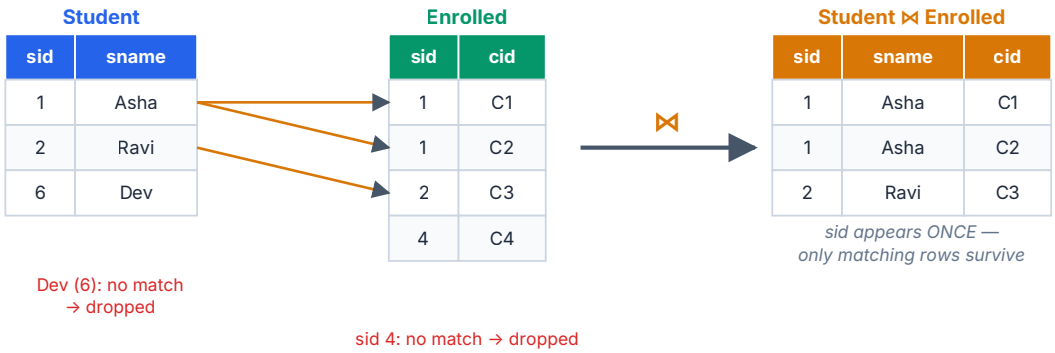


Fig. 18 — Only matching rows survive; tuples without a partner are dropped

Natural join — Student ⋈ Enrolled

The only common attribute is **sid**, so rows are matched on Student.sid = Enrolled.sid and sid appears once.

RELATIONAL ALGEBRA

Student ⋈ Enrolled

```sql
SELECT * FROM Student NATURAL JOIN Enrolled;
```

RESULT — 11 ROWS

sid	sname	age	dept	cid	grade
1	Asha	20	CSE	C1	A
1	Asha	20	CSE	C2	B
1	Asha	20	CSE	C5	A
2	Ravi	21	ECE	C3	B
2	Ravi	21	ECE	C1	C
3	Meena	22	CSE	C1	A
3	Meena	22	CSE	C2	A
3	Meena	22	CSE	C5	B
4	Karan	20	ME	C4	B
5	Sara	21	CSE	C1	B
5	Sara	21	CSE	C2	A

Equi-join with a filter

Question: name and grade of every student enrolled in C1.

RELATIONAL ALGEBRA

$\pi_{\text{sname, cid, grade}}(\sigma_{\text{cid='C1'}}(\text{Student} \bowtie_{\text{Student.sid = Enrolled.sid}} \text{Enrolled}))$

```sql
SELECT s.sname, e.cid, e.grade
FROM Student s JOIN Enrolled e ON s.sid = e.sid
WHERE e.cid = 'C1';
```

RESULT — 4 ROWS

sname	cid	grade
Asha	C1	A
Ravi	C1	C
Meena	C1	A
Sara	C1	B

⚠ **Careful — The natural-join trap**

Student and **Course** both have an attribute called **dept**, so Student ⋈ Course silently joins on dept = dept — it pairs every student with every course of the *same department*, which is almost never what you want:

```sql
SELECT COUNT(*) AS rows_returned FROM Student NATURAL JOIN Course;
```

RESULT — 1 ROW

rows_returned
12

Defences: (1) **project away** the clashing column before joining, (2) **rename** it with ρ, or (3) use an explicit theta join ($\bowtie_{...}$) / SQL's `JOIN … ON`.

> 🧠 **Yaad rakho**
>
> Natural join = **same naam wale columns pe automatic match**. Naam same par matlab alag ho to galat result — isliye explicit join safe hai.

3.2 Joining three relations

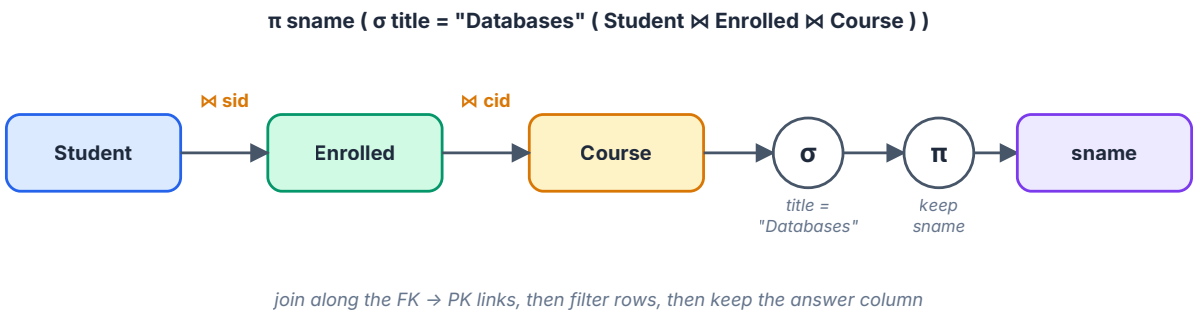


Fig. 19 — Join along the foreign-key links, then filter, then project the answer

Question: names of students enrolled in *Databases*. The path is Student → Enrolled → Course. We keep only the columns we need before joining, so **dept** cannot clash:

RELATIONAL ALGEBRA

$$\pi_{\text{sname}}(\sigma_{\text{title='Databases'}}(\pi_{\text{sid, sname}}(\text{Student}) \bowtie \text{Enrolled} \bowtie \pi_{\text{cid, title}}(\text{Course})))$$

```sql
SELECT DISTINCT s.sname
FROM Student s
JOIN Enrolled e ON s.sid = e.sid
JOIN Course c ON e.cid = c.cid
WHERE c.title = 'Databases';
```

RESULT — 4 ROWS

sname
Asha
Ravi
Meena
Sara

Question: students who earned an A in *AI*.

RELATIONAL ALGEBRA

$\pi_{sname}(\sigma_{title='AI' \wedge grade='A'}(\pi_{sid,\ sname}(Student) \bowtie Enrolled \bowtie \pi_{cid,\ title}(Course)))$

```sql
SELECT s.sname
FROM Student s
JOIN Enrolled e ON s.sid = e.sid
JOIN Course c ON e.cid = c.cid
WHERE c.title = 'AI' AND e.grade = 'A';
```

RESULT — 1 ROW

sname
Asha

⚠️ **Careful — What if we had used plain natural joins?**

Student ⋈ Enrolled ⋈ Course would also match **dept = dept**, so **Ravi** (ECE) disappears from the Databases answer because C1 is a CSE course:

```sql
SELECT DISTINCT sname
FROM Student NATURAL JOIN Enrolled NATURAL JOIN Course
WHERE title = 'Databases';
```

RESULT — 3 ROWS

sname
Asha
Meena
Sara

⭐ **Exam tip — Join order**

Joins are **associative and commutative**, so the *result* does not depend on the order — only the speed does. The optimizer picks the order (§3.10).

3.3 Outer joins

📘 **Definition**

An inner join drops tuples with no partner. An **outer join** keeps them and pads the missing side with **NULL**: **left outer** keeps all tuples of the left relation, **right outer** all of the right, **full outer** all of both. Written here as $⟕$, $⟖$, $⟗$.

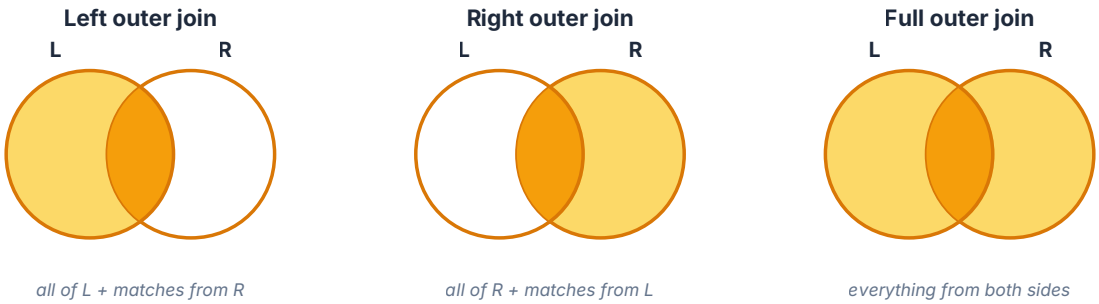


Fig. 20 — The three outer joins keep unmatched tuples from one or both sides

Left outer example: every student, with their C4 enrolment if they have one.

RELATIONAL ALGEBRA

$$\text{Student} \;⟕_{L}\; \sigma_{cid='C4'}(\text{Enrolled})$$

```sql
SELECT s.sid, s.sname, e.cid, e.grade
FROM Student s
LEFT JOIN (SELECT * FROM Enrolled WHERE cid = 'C4') e ON s.sid = e.sid;
```

RESULT — 6 ROWS

sid	sname	cid	grade
1	Asha	NULL	NULL
2	Ravi	NULL	NULL
3	Meena	NULL	NULL
4	Karan	C4	B
5	Sara	NULL	NULL
6	Dev	NULL	NULL

Full outer example: students 1–2 against the C3/C4 enrolments. Asha has no match, student 4 has no matching row on the left — both survive.

RELATIONAL ALGEBRA

$$\sigma_{sid\in\{1,2\}}(\text{Student}) \;⟗_{F}\; \sigma_{cid='C3' \vee cid='C4'}(\text{Enrolled})$$

```sql
SELECT l.sid AS l_sid, l.sname, r.sid AS r_sid, r.cid
FROM (SELECT * FROM Student WHERE sid IN (1,2)) l
FULL JOIN (SELECT * FROM Enrolled WHERE cid IN ('C3','C4')) r
  ON l.sid = r.sid;
```

RESULT — 3 ROWS

l_sid	sname	r_sid	cid
1	Asha	NULL	NULL
2	Ravi	2	C3
NULL	NULL	4	C4

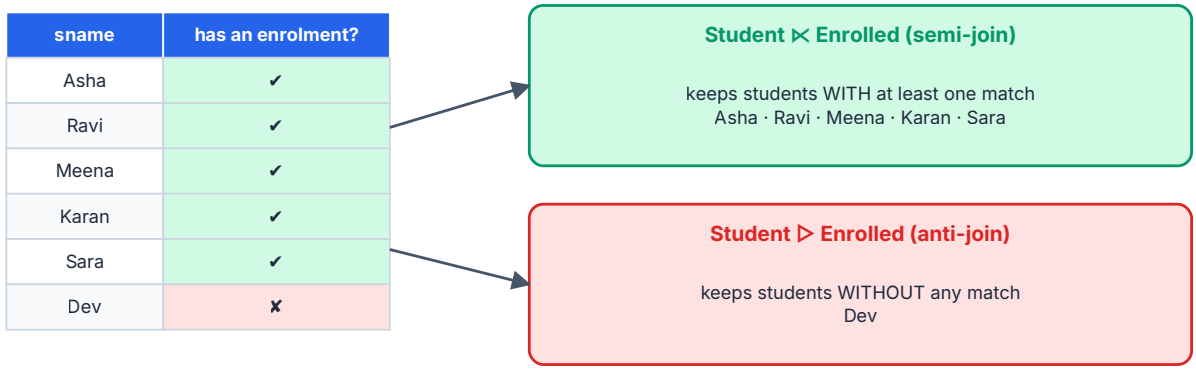
🏭 Real-life use case

Reports like "all customers with their orders — including customers who never ordered" need a **left outer join**; an inner join would silently hide the customers with no orders.

3.4 Semi-join (⋉) and anti-join (▷)

📘 Definition

R ⋉ S (semi-join) returns the tuples of R that **have** a matching tuple in S — and only R's columns: $R \ltimes S = \pi_{attrs(R)}(R \bowtie S)$. **R ▷ S** (anti-join) returns the tuples of R that have **no** match: $R \triangleright S = R - (R \ltimes S)$.



Neither adds Enrolled columns — they only FILTER Student

Fig. 21 — Semi-join keeps students with a match; anti-join keeps those without

RELATIONAL ALGEBRA

Student ⋉ Enrolled

```sql
SELECT * FROM Student s
WHERE EXISTS (SELECT 1 FROM Enrolled e WHERE e.sid = s.sid);
```

RESULT — 5 ROWS

sid	sname	age	dept
1	Asha	20	CSE
2	Ravi	21	ECE
3	Meena	22	CSE
4	Karan	20	ME
5	Sara	21	CSE

RELATIONAL ALGEBRA

Student ▷ Enrolled

```sql
SELECT * FROM Student s
WHERE NOT EXISTS (SELECT 1 FROM Enrolled e WHERE e.sid = s.sid);
```

RESULT — 1 ROW

sid	sname	age	dept
6	Dev	23	ECE

⭐ **Exam tip — SQL form**

Semi-join = `WHERE EXISTS (…)`, anti-join = `WHERE NOT EXISTS (…)`. Prefer `NOT EXISTS` over `NOT IN`: a single NULL inside the `NOT IN` list makes the whole condition unknown and returns no rows.

3.5 Writing "not", "both" and "either" queries

Most mistakes come from translating English words into the wrong operator. Use this map first.

English cue	Reach for	Example
who has / has taken / with	⋈ then π	students enrolled in Databases
and / both / as well as	∩ or self-join	enrolled in C1 AND C2
or / either / at least one of	∪ or ∨ inside σ	enrolled in C1 OR C2
not / never / without / only	− or ▷ anti-join	students NOT in C2
all / every / each	÷ division	enrolled in ALL CSE courses
count / average / max / per	γ grouping	students per department
older than / same as another	ρ + self-join	pairs in the same department

Fig. 22 — English cue → the operator to reach for

"Not" → difference (or anti-join)

Question: names of students **not** enrolled in C2.

RELATIONAL ALGEBRA

$\pi_{sname}(Student \bowtie (\pi_{sid}(Student) - \pi_{sid}(\sigma_{cid='C2'}(Enrolled))))$

```sql
SELECT sname FROM Student
WHERE sid IN (SELECT sid FROM Student
              EXCEPT
              SELECT sid FROM Enrolled WHERE cid = 'C2');
```

RESULT — 3 ROWS

sname
Ravi
Karan
Dev

⚠️ **Careful — The classic wrong answer**

Writing $\sigma_{cid\neq'C2'}$ is wrong: it returns students who have *some other* course, not students who lack C2. Asha takes C2 yet still appears, and Dev (no enrolment at all) is missing:

```sql
SELECT DISTINCT s.sname
FROM Student s JOIN Enrolled e ON s.sid = e.sid
WHERE e.cid <> 'C2';
```

RESULT — 5 ROWS

sname
Asha
Ravi
Meena
Karan
Sara

"Not" always means **subtract the ones that do** from everyone.

"Both" → intersection or self-join

Question: students enrolled in **both** C1 and C2 (a single row cannot hold two different cid values, so we need two copies of ENROLLED):

RELATIONAL ALGEBRA

$$\pi_{e1.sid}(\sigma_{e1.cid='C1' \wedge e2.cid='C2'}(\rho_{e1}(Enrolled) \bowtie_{e1.sid = e2.sid} \rho_{e2}(Enrolled)))$$

```sql
SELECT DISTINCT e1.sid
FROM Enrolled e1 JOIN Enrolled e2 ON e1.sid = e2.sid
WHERE e1.cid = 'C1' AND e2.cid = 'C2';
```

RESULT — 3 ROWS

sid
1
3
5

"Either" → union (or ∨)

Enrolled in C1 **or** C2: $\pi_{sid}(\sigma_{cid='C1' \vee cid='C2'}(Enrolled))$ — the ∨ inside one selection is simpler than a union (Unit II gave the union form).

3.6 Self-join

📘 **Definition**

A **self-join** joins a relation with **itself**. Rename it twice with ρ so the two copies have different names, then join them.

Question: pairs of students from the same department (each pair once).

RELATIONAL ALGEBRA

$$\pi_{s1.sname,\ s2.sname}(\rho_{s1}(Student) \bowtie_{s1.dept = s2.dept \wedge s1.sid < s2.sid} \rho_{s2}(Student))$$

```sql
SELECT s1.sname AS student1, s2.sname AS student2
FROM Student s1 JOIN Student s2
  ON s1.dept = s2.dept AND s1.sid < s2.sid;
```

RESULT — 4 ROWS

student1	student2
Asha	Meena
Asha	Sara
Ravi	Dev
Meena	Sara

⭐ **Exam tip — The < trick**

Using **s1.sid < s2.sid** instead of ≠ removes both (x, x) and the mirrored pair (y, x), so each pair appears once.

3.7 Division (÷) — the "for ALL" operator

Definition

Let **A(x, y)** and **B(y)**. **A ÷ B** returns the values of **x** that are paired in A with **every** y in B. B's attributes must be a subset of A's, and the result has the remaining attributes (A − B).

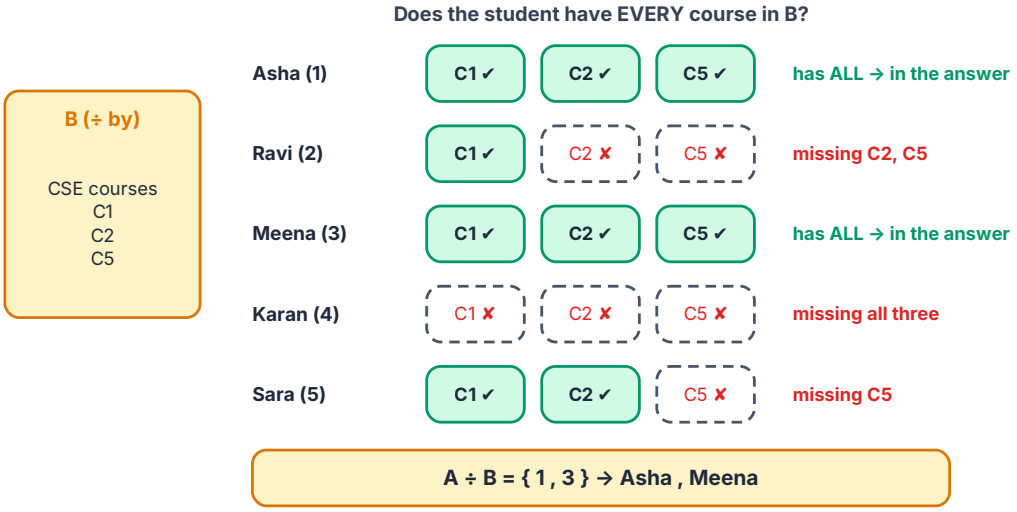


Fig. 23 — Keep only the students who have every course in B

Question: students enrolled in **all CSE courses**.

RELATIONAL ALGEBRA

$$A \leftarrow \pi_{sid,\ cid}(\text{Enrolled})\ B \leftarrow \pi_{cid}(\ \sigma_{dept='CSE'}(\text{Course})\ )\ A \div B$$

Division without ÷ — the four-step derivation

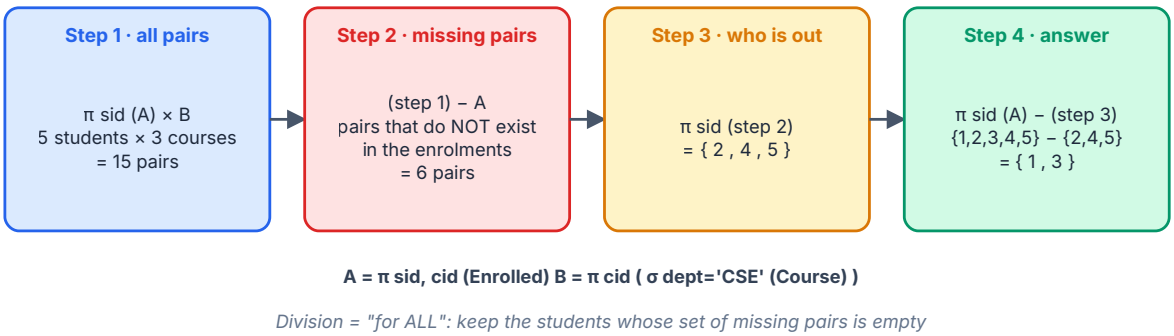


Fig. 24 — Division built from π, ×, and −

RELATIONAL ALGEBRA

$$\pi_{sid}(A) - \pi_{sid}(\ (\ \pi_{sid}(A) \times B\ ) - A\ )$$

The inner part lists the **missing pairs** — a student and a CSE course that the student is *not* enrolled in:

```sql
SELECT a.sid, b.cid
FROM (SELECT DISTINCT sid FROM Enrolled) a
CROSS JOIN (SELECT cid FROM Course WHERE dept = 'CSE') b
EXCEPT
SELECT sid, cid FROM Enrolled;
```

RESULT — 6 ROWS

sid	cid
2	C2
2	C5
4	C1
4	C2
4	C5
5	C5

Students 2, 4 and 5 appear, so they are disqualified; the remaining students are the answer. In SQL, **division** is the double `NOT EXISTS` pattern ("no course exists that the student is missing"):

```sql
SELECT s.sid, s.sname FROM Student s
WHERE NOT EXISTS (
  SELECT * FROM Course c WHERE c.dept = 'CSE'
  AND NOT EXISTS (
    SELECT * FROM Enrolled e WHERE e.sid = s.sid AND e.cid = c.cid));
```

RESULT — 2 ROWS

sid	sname
1	Asha
3	Meena

A second SQL way counts: the student must have as many matching courses as there are CSE courses.

```sql
SELECT sid FROM Enrolled
WHERE cid IN (SELECT cid FROM Course WHERE dept = 'CSE')
GROUP BY sid
HAVING COUNT(DISTINCT cid) = (SELECT COUNT(*) FROM Course WHERE dept = 'CSE');
```

RESULT — 2 ROWS

sid
1
3

⚠️ **Careful — Division gotchas**

- Dev (no enrolment) is correctly **excluded**; Karan took one non-CSE course and is excluded too.
- Division is **not commutative** — A ÷ B and B ÷ A are different (and usually B ÷ A is not even defined).
- If B is **empty**, A ÷ B returns every x in A — "all of nothing" is vacuously true.

🧠 **Yaad rakho**

"**Sabhi / har ek / all / every**" dikhe to ÷ socho. Yaad rakho: **sid ke saath saare cid** hone chahiye.

3.8 Grouping and aggregation (γ)

Definition

Aggregation computes a value from a set of tuples (**count, sum, avg, min, max**). **Grouping** first splits the relation into groups sharing the same value of the grouping attributes. Notation: $\gamma_{groupAttrs\ ;\ aggregates}(R)$, written G in some books.

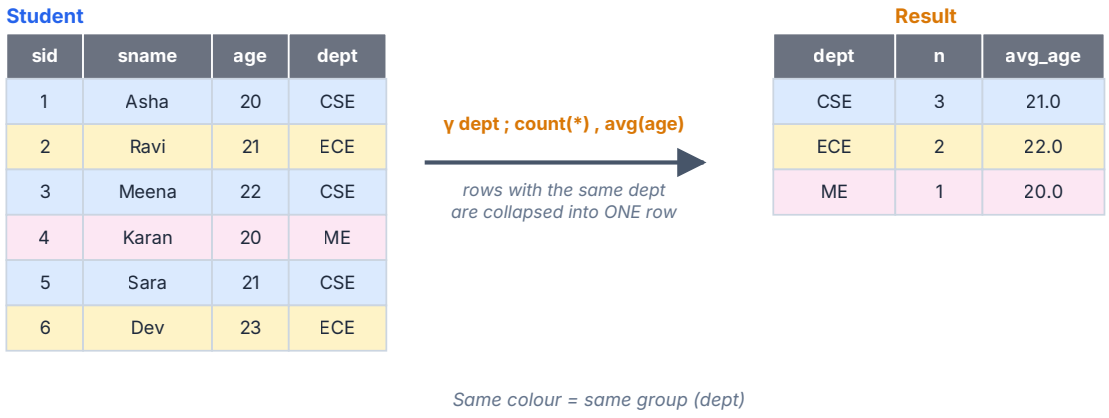


Fig. 25 — Rows with the same dept collapse into one result row

RELATIONAL ALGEBRA

$\gamma_{dept\ ;\ count(*)\rightarrow n,\ avg(age)\rightarrow avg_age}(Student)$

```sql
SELECT dept, COUNT(*) AS n, AVG(age) AS avg_age
FROM Student GROUP BY dept ORDER BY dept;
```

RESULT — 3 ROWS

dept	n	avg_age
CSE	3	21.0
ECE	2	22.0
ME	1	20.0

A condition on the **result** of an aggregate (SQL's `HAVING`) is just a σ applied **after** γ:

RELATIONAL ALGEBRA

$\sigma_{n\geq 2}(\ \gamma_{dept\ ;\ count(*)\rightarrow n}(Student)\ )$

```sql
SELECT dept, COUNT(*) AS n
FROM Student GROUP BY dept
HAVING COUNT(*) >= 2 ORDER BY dept;
```

RESULT — 2 ROWS

dept	n
CSE	3
ECE	2

Combine with an outer join to count zero correctly — Dev has no courses and still shows **0**, because `COUNT(e.cid)` ignores NULLs:

RELATIONAL ALGEBRA

$$\gamma_{\text{sid, sname ; count(cid)}\rightarrow\text{courses}}(\text{Student} \bowtie_{L} \text{Enrolled})$$

```sql
SELECT s.sname, COUNT(e.cid) AS courses
FROM Student s LEFT JOIN Enrolled e ON s.sid = e.sid
GROUP BY s.sid, s.sname ORDER BY s.sid;
```

RESULT — 6 ROWS

sname	courses
Asha	3
Ravi	2
Meena	3
Karan	1
Sara	2
Dev	0

Largest value — with and without aggregation

Question: the oldest student. Using γ: compute max(age), then select the matching row.

RELATIONAL ALGEBRA

$$\pi_{\text{sname}}(\sigma_{\text{age}=m}(\text{Student} \times \gamma_{\text{max(age)}\rightarrow m}(\text{Student})))$$

```sql
SELECT sname FROM Student
WHERE age = (SELECT MAX(age) FROM Student);
```

RESULT — 1 ROW

sname
Dev

Pure algebra (no γ) uses **difference**: remove every student who has someone older.

RELATIONAL ALGEBRA

$$\pi_{\text{sname}}(\text{Student}) - \pi_{\text{s1.sname}}(\rho_{\text{s1}}(\text{Student}) \bowtie_{\text{s1.age} < \text{s2.age}} \rho_{\text{s2}}(\text{Student}))$$

```sql
SELECT sname FROM Student
EXCEPT
SELECT s1.sname FROM Student s1 JOIN Student s2 ON s1.age < s2.age;
```

RESULT — 1 ROW

sname
Dev

3.9 Extended operators and NULLs

Operator	Meaning	SQL
Generalized projection $\pi_{sname,\ age+1 \to next_age}$	Projection may contain arithmetic expressions and renames	`SELECT sname, age + 1 AS next_age`
Assignment $R \leftarrow expr$	Names an intermediate result	`WITH R AS (…)`
Sort $\tau_{age\ desc}(R)$	Orders the output (a presentation step — relations themselves are unordered)	`ORDER BY age DESC`

RELATIONAL ALGEBRA

$$\pi_{sname,\ age+1 \to next_age}(\sigma_{dept='CSE'}(Student))$$

SQL

```
SELECT sname, age + 1 AS next_age FROM Student WHERE dept = 'CSE';
```

RESULT — 3 ROWS

sname	next_age
Asha	21
Meena	23
Sara	22

Three-valued logic

Comparisons with NULL give **UNKNOWN** (U). A selection keeps a row only if the condition is TRUE.

A	B	A ∧ B	A ∨ B	¬A
T	U	U	T	F
F	U	F	U	T
U	U	U	U	U

3.10 Equivalent expressions and optimization

Different algebra expressions can give the **same** answer at very different cost. The optimizer applies **equivalence rules** to find a cheaper expression.

#	Rule	Why it helps
1	$\sigma_{\theta 1 \wedge \theta 2}(R) = \sigma_{\theta 1}(\sigma_{\theta 2}(R))$ — selections cascade and commute	Apply the most selective condition first
2	$\pi_A(\pi_B(R)) = \pi_A(R)$ when $A \subseteq B$	Drop redundant projections
3	$\sigma_\theta(R \times S) = R \bowtie_\theta S$	Avoid building the huge product
4	$\sigma_\theta(R \bowtie S) = \sigma_\theta(R) \bowtie S$ if θ uses only R's attributes	**Push selection down** — shrink inputs before the join
5	$R \bowtie S = S \bowtie R$, $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$	Choose the cheapest join order
6	σ distributes over ∪, ∩, −	Filter each branch early

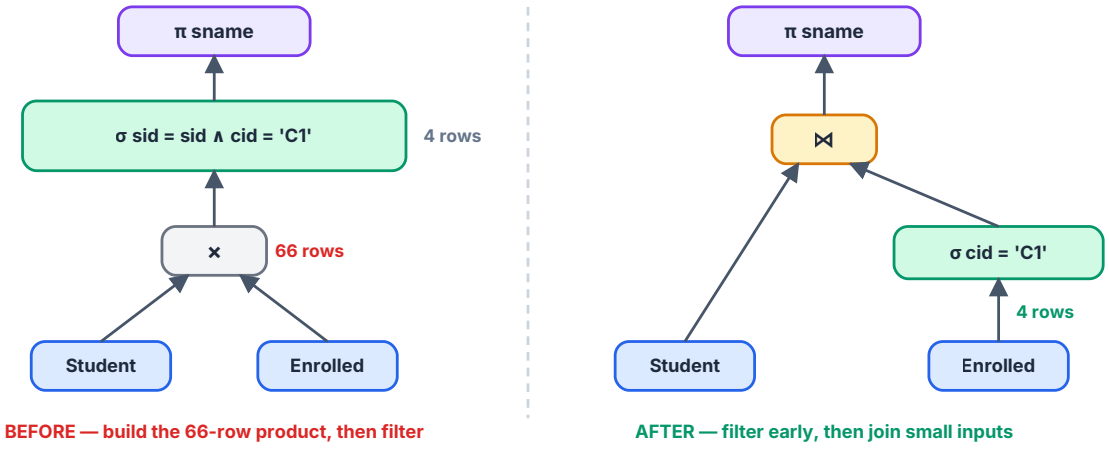


Fig. 26 — Pushing the selection below the join: same answer, far fewer rows to process

For *"names of students in C1"* the left plan builds **6 × 11 = 66** product rows and then filters; the right plan first cuts ENROLLED to **4 rows** and joins those — so the join compares only **6 × 4 = 24** pairs instead of 66 — and the gap grows enormously with real data sizes.

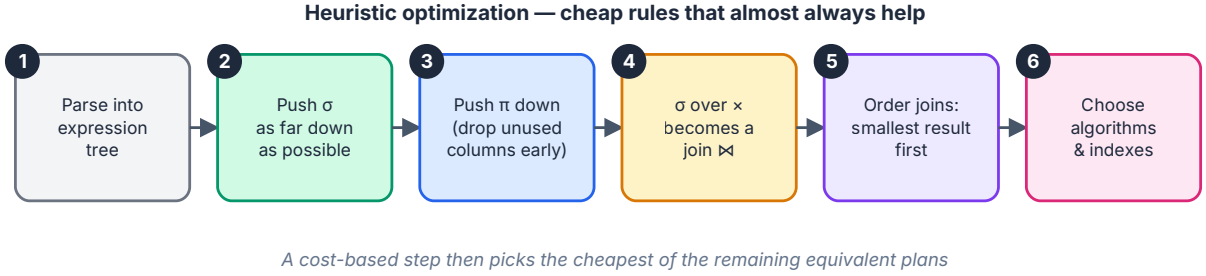


Fig. 27 — A heuristic optimizer's checklist

> ⭐ **Exam tip**
>
> Remember the two golden rules of heuristic optimization: **"push selections down"** and **"push projections down"** — do the reducing operations as early as possible, the expanding ones (× and ⋈) as late as possible.

3.11 Algebra ⇄ SQL translation table

Relational algebra	SQL
$\sigma_{\theta}(R)$	`SELECT * FROM R WHERE θ`
$\pi_{A}(R)$	`SELECT DISTINCT A FROM R`
$\rho_{S}(R)$	`FROM R AS S`
$R \cup S \cdot R \cap S \cdot R - S$	`UNION` · `INTERSECT` · `EXCEPT` (Oracle: `MINUS`)
$R \times S$	`FROM R CROSS JOIN S`
$R \bowtie_{\theta} S$	`FROM R JOIN S ON θ`
$R \bowtie S$ (natural)	`FROM R NATURAL JOIN S` (or `JOIN … USING`)
$\bowtie_{L} \cdot \bowtie_{R} \cdot \bowtie_{F}$	`LEFT JOIN` · `RIGHT JOIN` · `FULL JOIN`
$R \ltimes S \cdot R \triangleright S$	`WHERE EXISTS (…)` · `WHERE NOT EXISTS (…)`
$R \div S$	double `NOT EXISTS`, or `GROUP BY … HAVING COUNT(DISTINCT …) = …`
$\gamma_{G\ ;\ agg}(R)$	`SELECT G, agg FROM R GROUP BY G` (+ `HAVING`)
$\tau_{A}(R)$	`ORDER BY A`

❓ Likely exam questions — Unit III

- Differentiate theta, equi and natural join with examples.
- What is an outer join? Explain left, right and full with an example.
- Explain semi-join and anti-join; how are they written in SQL?
- Write the algebra for "students not enrolled in C2" and explain why $\sigma_{cid \neq 'C2'}$ is wrong.
- Define division; derive it using π, × and −; give an example.
- Explain grouping and aggregation with notation; how is HAVING expressed?
- How do you find the maximum without an aggregate function?
- List equivalence rules; explain "push selection down" with a diagram.
- Why can natural join give wrong results? How do you avoid it?

UNIT IV

Tuple Relational Calculus (TRC)

Algebra vs calculus | TRC syntax | Free & bound variables | ∃ and ∀ | "Not" and "all" queries | Safety | TRC → SQL

4.1 From algebra to calculus

Definition

Relational calculus is a **declarative** query language based on predicate logic: you describe the properties of the tuples you want, and say nothing about how to find them. Its two forms are **tuple relational calculus (TRC)**, whose variables stand for whole tuples, and **domain relational calculus (DRC)**, whose variables stand for single values.

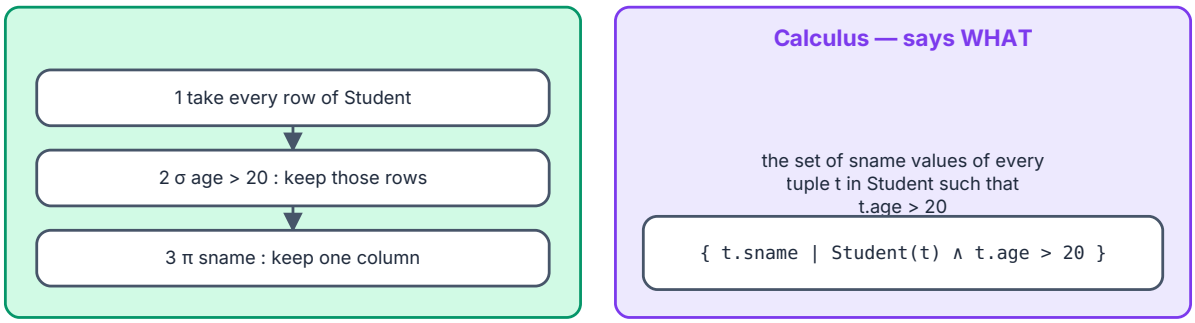


Fig. 28 — Same question, two styles: a recipe (algebra) and a description (calculus)

Algebra tells the database *which operators to apply, in which order*. Calculus only states *what the answer must satisfy*. In this unit we write queries in TRC; Unit V does DRC.

RELATIONAL ALGEBRA

$\pi_{sname}(\sigma_{age>20}(Student))$

TUPLE CALCULUS

$\{ t.sname \mid Student(t) \wedge t.age > 20 \}$

Yaad rakho

Algebra: **"pehle ye karo, phir ye"**. Calculus: **"mujhe wo tuples do jinke liye ye condition sach ho"**.

4.2 TRC syntax

📘 Definition

A TRC query has the form { t | P(t) }: **the set of all tuples t such that the formula P is true for t**. A **tuple variable** t takes whole tuples as values. A **range** atom Student(t) says t ranges over the STUDENT relation, and t.age picks one attribute of t.

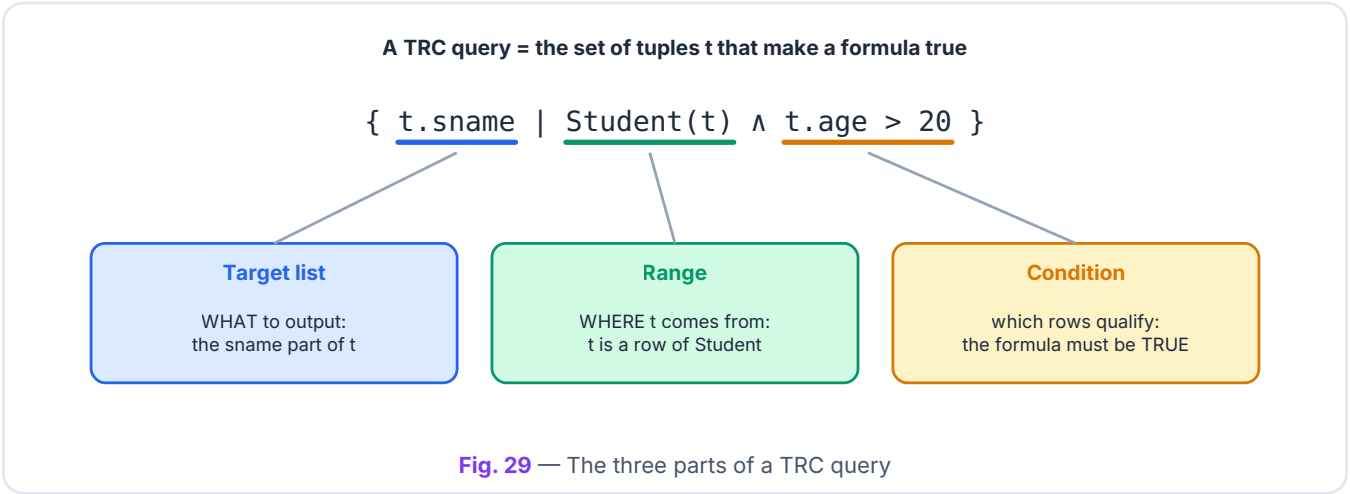


Fig. 29 — The three parts of a TRC query

Building block	Meaning
R(t)	t is a tuple of relation R (the **range**)
t.A θ s.B or t.A θ c	Comparison of attributes, or of an attribute with a constant; θ is one of = ≠ < ≤ > ≥
¬P · P ∧ Q · P ∨ Q · P ⇒ Q	not · and · or · implies (P ⇒ Q ≡ ¬P ∨ Q)
∃t (P(t))	there **exists** at least one tuple t for which P is true
∀t (P(t))	for **all** tuples t, P is true

Precedence (highest first): comparisons, ¬, ∧, ∨, ⇒. Use brackets freely — clarity beats cleverness.

⭐ Exam tip — Convention

In the target list we write t.sname (or t[sname]) to output a column; writing just t returns the whole tuple.

4.3 Free and bound variables

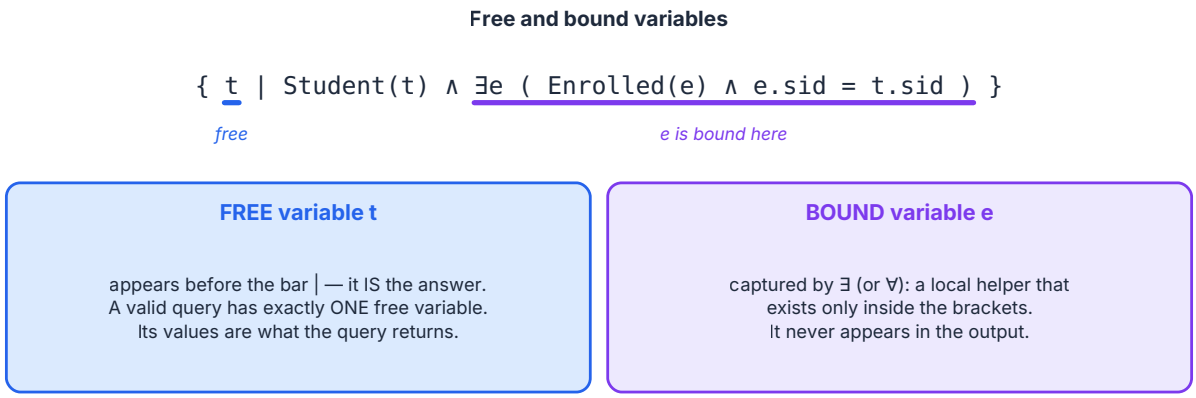


Fig. 30 — t is free (it is the answer); e is bound by the quantifier

- A **free** variable is not captured by any quantifier. The variable(s) before the bar | must be free — they are what the query returns.
- A **bound** variable is introduced by ∃ or ∀; it is a local helper and exists only inside its brackets.
- A valid TRC query has **exactly the result variable free**; every other variable must be bound.

⚠️ **Careful — A common slip**

Using a variable like e in the formula but never declaring it with ∃e leaves a second free variable, which makes the query ill-formed.

4.4 Selection and projection

Question: all students older than 20 (whole tuples — selection).

TUPLE CALCULUS

$\{ t \mid Student(t) \wedge t.age > 20 \}$

SAME IN ALGEBRA

$\sigma_{age>20}(Student)$

```
SQL
SELECT * FROM Student WHERE age > 20;
```

RESULT — 4 ROWS

sid	sname	age	dept
2	Ravi	21	ECE
3	Meena	22	CSE
5	Sara	21	CSE
6	Dev	23	ECE

Question: names of CSE students (a column — projection).

TUPLE CALCULUS

$$\{ t.sname \mid Student(t) \wedge t.dept = 'CSE' \}$$

SAME IN ALGEBRA

$$\pi_{sname}(\sigma_{dept='CSE'}(Student))$$

```sql
SELECT DISTINCT sname FROM Student WHERE dept = 'CSE';
```

RESULT — 3 ROWS

sname
Asha
Meena
Sara

The target list does the work of π, and the comparison does the work of σ.

4.5 Joins with ∃

Calculus has no join operator. Instead you ask: *"there exists a related tuple in the other relation."*

Question: names of students enrolled in C1.

TUPLE CALCULUS

$$\{ t.sname \mid Student(t) \wedge \exists e \, (Enrolled(e) \wedge e.sid = t.sid \wedge e.cid = 'C1') \}$$

The condition $e.sid = t.sid$ links the two tuples — it plays the role of the join condition. In SQL the ∃ becomes `EXISTS`:

```sql
SELECT sname FROM Student t
WHERE EXISTS (SELECT 1 FROM Enrolled e
              WHERE e.sid = t.sid AND e.cid = 'C1');
```

RESULT — 4 ROWS

sname
Asha
Ravi
Meena
Sara

Tuple relational calculus	SQL
{ t.sname | ... } (target list)	SELECT sname
Student(t) (range)	FROM Student t
formula (the condition)	WHERE ...
∃e (Enrolled(e) ∧ e.sid = t.sid)	EXISTS (SELECT 1 FROM Enrolled e WHERE e.sid = t.sid)
¬∃e (...)	NOT EXISTS (...)
∀e (...) ≡ ¬∃e ¬(...)	NOT EXISTS (... AND NOT (...))
∧ ∨ ¬	AND OR NOT

Fig. 31 — Each TRC construct has a direct SQL counterpart

Question: names of students enrolled in *Databases* — two related tuples are needed, so two ∃:

TUPLE CALCULUS

{ t.sname | Student(t) ∧ ∃e ∃c (Enrolled(e) ∧ Course(c) ∧ e.sid = t.sid ∧ e.cid = c.cid ∧ c.title = 'Databases') }

```sql
SELECT DISTINCT s.sname
FROM Student s
JOIN Enrolled e ON s.sid = e.sid
JOIN Course c ON e.cid = c.cid
WHERE c.title = 'Databases';
```

RESULT — 4 ROWS

sname
Asha
Ravi
Meena
Sara

💡 **Why it matters — Why this matters**

SQL is built on tuple calculus: **FROM** declares the tuple variables (ranges), **WHERE** holds the formula, **SELECT** is the target list. Reading TRC makes subqueries and EXISTS feel natural.

4.6 The two quantifiers

📘 **Definition**

∃t P(t) is true when **at least one** tuple makes P true (like an OR over all tuples). ∀t P(t) is true when **every** tuple makes P true (like an AND over all tuples).

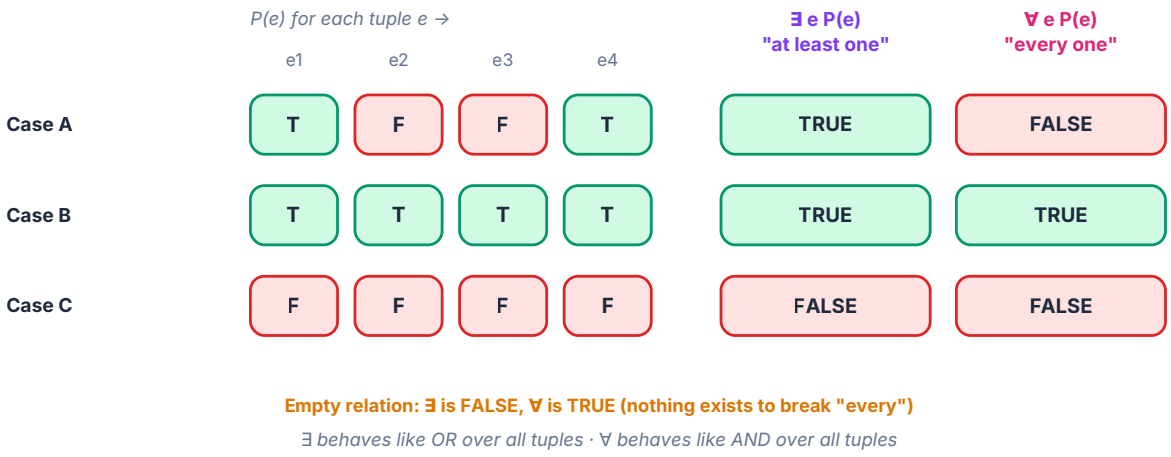


Fig. 32 — ∃ needs one T; ∀ needs all T

Equivalence	In words
¬∃t P(t) ≡ ∀t ¬P(t)	"nothing satisfies P" = "everything fails P"
¬∀t P(t) ≡ ∃t ¬P(t)	"not everything satisfies P" = "something fails P"
∀t (P ⇒ Q) ≡ ¬∃t (P ∧ ¬Q)	"every P is a Q" = "no P is a non-Q"
P ⇒ Q ≡ ¬P ∨ Q	implication as a disjunction
¬(P ∧ Q) ≡ ¬P ∨ ¬Q · ¬(P ∨ Q) ≡ ¬P ∧ ¬Q	De Morgan's laws

⚠️ **Careful — Vacuous truth — ∀ over nothing is TRUE**

Question: students *all of whose* enrolments have grade A. The natural formula is:

TUPLE CALCULUS

{ t.sname | Student(t) ∧ ∀e ((Enrolled(e) ∧ e.sid = t.sid) ⇒ e.grade = 'A') }

```sql
SELECT sname FROM Student s
WHERE NOT EXISTS (SELECT 1 FROM Enrolled e
                  WHERE e.sid = s.sid AND e.grade <> 'A');
```

RESULT — 1 ROW

sname
Dev

Nobody has only A's, yet **Dev** appears — he has *no* enrolments, so "every enrolment is an A" is vacuously true. Add an existence test to fix it:

TUPLE CALCULUS

{ t.sname | Student(t) ∧ ∃e2 (Enrolled(e2) ∧ e2.sid = t.sid) ∧ ∀e ((Enrolled(e) ∧ e.sid = t.sid) ⇒ e.grade = 'A') }

SQL

```
SELECT sname FROM Student s
WHERE EXISTS (SELECT 1 FROM Enrolled e WHERE e.sid = s.sid)
  AND NOT EXISTS (SELECT 1 FROM Enrolled e
                  WHERE e.sid = s.sid AND e.grade <> 'A');
```

RESULT — 0 ROWS

sname

The corrected answer is empty — correct for our data.

4.7 "Not" queries with ¬∃

Question: names of students **not** enrolled in C2.

TUPLE CALCULUS

{ t.sname | Student(t) ∧ ¬∃e (Enrolled(e) ∧ e.sid = t.sid ∧ e.cid = 'C2') }

SQL

```
SELECT sname FROM Student t
WHERE NOT EXISTS (SELECT 1 FROM Enrolled e
                  WHERE e.sid = t.sid AND e.cid = 'C2');
```

RESULT — 3 ROWS

sname
Ravi
Karan
Dev

Question: students with **no enrolment at all** (compare with the anti-join of Unit III).

TUPLE CALCULUS

{ t.sname | Student(t) ∧ ¬∃e (Enrolled(e) ∧ e.sid = t.sid) }

SQL

```
SELECT sname FROM Student s
WHERE NOT EXISTS (SELECT 1 FROM Enrolled e WHERE e.sid = s.sid);
```

RESULT — 1 ROW

sname
Dev

🧠 **Yaad rakho**

"**nahi / kabhi nahi / koi nahi**" = **¬∃**. Pehle "wo jo karte hain" likho, phir uske aage ¬∃ laga do.

4.8 "For all" queries with ∀ (the division pattern)

Question: students enrolled in **every CSE course**. Say it with ∀ and ⇒, then flip it to ¬∃ — the second form maps straight to SQL.

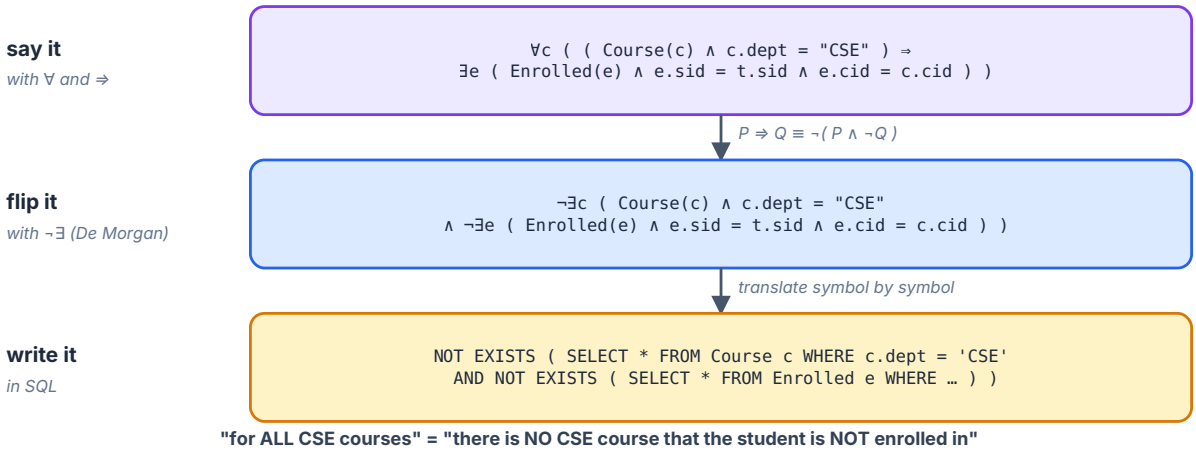


Fig. 33 — Say it, flip it, write it

TUPLE CALCULUS

$$\{ t.sname \mid Student(t) \wedge \forall c ((Course(c) \wedge c.dept = 'CSE') \Rightarrow \exists e (Enrolled(e) \wedge e.sid = t.sid \wedge e.cid = c.cid)) \}$$

SQL

```
SELECT s.sid, s.sname FROM Student s
WHERE NOT EXISTS (
  SELECT * FROM Course c WHERE c.dept = 'CSE'
  AND NOT EXISTS (
    SELECT * FROM Enrolled e WHERE e.sid = s.sid AND e.cid = c.cid));
```

RESULT — 2 ROWS

sid	sname
1	Asha
3	Meena

Exactly Asha and Meena — the same answer the algebra division gave in Unit III. The ∀ pattern is **always**: *for every c in the set, there exists a connecting tuple.*

4.9 Safety of expressions

📘 **Definition**

A TRC expression is **safe** if it is guaranteed to return a **finite** set built only from values that appear in the database (or the query). Only safe expressions are legitimate queries.

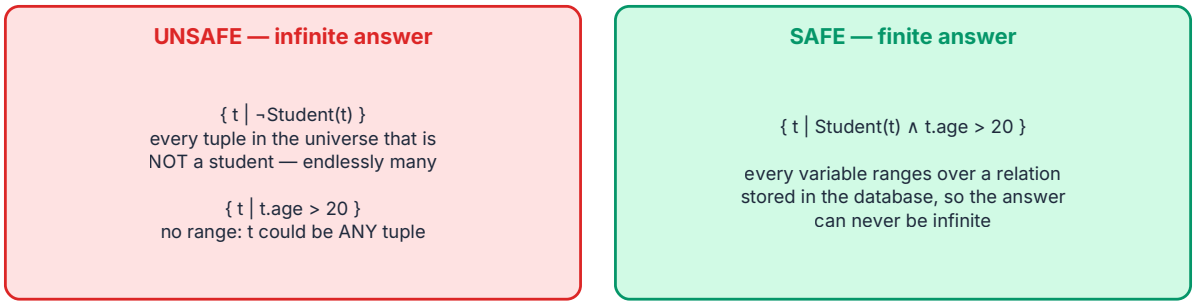


Fig. 34 — Unsafe expressions describe infinite sets; safe ones are tied to stored relations

The expression { t | ¬Student(t) } asks for "every tuple that is not a student" — infinitely many possible tuples — so no database could return it. **Rule:** every variable in the target list must be restricted by a range atom R(t) (or equated to something that is), so its values come from stored data.

> ⭐ **Exam tip**
>
> "Why must calculus expressions be safe?" → because an unsafe expression can denote an infinite relation, and relational algebra (which always produces finite results) could not express it. Safety is what makes calculus equivalent to algebra (Codd's theorem, Unit V).

4.10 Union, intersection and difference in TRC

Set operations become logical connectives: **∪ → ∨**, **∩ → ∧**, **− → ∧ ¬**. Using the C1 / C2 enrolments of Unit II (output: student ids):

UNION — C1 OR C2

$$\{ t.sid \mid Student(t) \wedge (\exists e (Enrolled(e) \wedge e.sid = t.sid \wedge e.cid = 'C1') \vee \exists e (Enrolled(e) \wedge e.sid = t.sid \wedge e.cid = 'C2')) \}$$

```sql
SELECT sid FROM Enrolled WHERE cid = 'C1'
UNION
SELECT sid FROM Enrolled WHERE cid = 'C2';
```

RESULT — 4 ROWS

sid
1
2
3
5

INTERSECTION — C1 AND C2

{ t.sid | Student(t) ∧ ∃e (Enrolled(e) ∧ e.sid = t.sid ∧ e.cid = 'C1') ∧ ∃e (Enrolled(e) ∧ e.sid = t.sid ∧ e.cid = 'C2') }

```sql
SELECT sid FROM Enrolled WHERE cid = 'C1'
INTERSECT
SELECT sid FROM Enrolled WHERE cid = 'C2';
```

RESULT — 3 ROWS

sid
1
3
5

DIFFERENCE — C1 BUT NOT C2

{ t.sid | Student(t) ∧ ∃e (Enrolled(e) ∧ e.sid = t.sid ∧ e.cid = 'C1') ∧ ¬∃e (Enrolled(e) ∧ e.sid = t.sid ∧ e.cid = 'C2') }

```sql
SELECT sid FROM Enrolled WHERE cid = 'C1'
EXCEPT
SELECT sid FROM Enrolled WHERE cid = 'C2';
```

RESULT — 1 ROW

sid
2

4.11 Algebra ⇄ TRC dictionary

Relational algebra	Tuple calculus	SQL
$\sigma_{\theta}(R)$	{ t | R(t) ∧ θ }	`WHERE θ`
$\pi_{A}(R)$	target list { t.A | … }	`SELECT DISTINCT A`
R ∪ S	R(t) ∨ S(t)	`UNION`
R ∩ S	R(t) ∧ S(t)	`INTERSECT`
R − S	R(t) ∧ ¬S(t)	`EXCEPT`
R × S	two ranges: R(r) ∧ S(s)	`FROM R, S`
R ⋈ S	R(r) ∧ ∃s (S(s) ∧ r.A = s.A)	`JOIN … ON` / `EXISTS`
R ÷ S	∀s (S(s) ⇒ ∃r (…))	double `NOT EXISTS`

❓ Likely exam questions — Unit IV

- Differentiate relational algebra and relational calculus.
- Explain the general form of a TRC query; what are tuple variables?
- Explain free and bound variables with an example.
- Explain the ∃ and ∀ quantifiers; state the equivalence between them.
- Write TRC for: students not in C2; students in all CSE courses.
- What is a safe expression? Give an unsafe example and say why it is unsafe.
- Show how SQL maps to TRC (SELECT, FROM, WHERE, EXISTS).
- Express union, intersection and difference in TRC.

UNIT V

Domain Calculus, Codd's Theorem & the Query-Writing Workbook

DRC | TRC vs DRC | Codd's theorem | Expressive power | QBE | Method | Master workbook | Practice

5.1 Domain relational calculus (DRC)

Definition

In **DRC** a variable stands for a **single value** from an attribute's domain, not a whole tuple. A query has the form { <x1, ..., xn> | P(x1, ..., xn) } — the set of value-lists that make the formula true. A **membership atom** <s, n, a, d> ∈ Student has one position per attribute, in schema order.

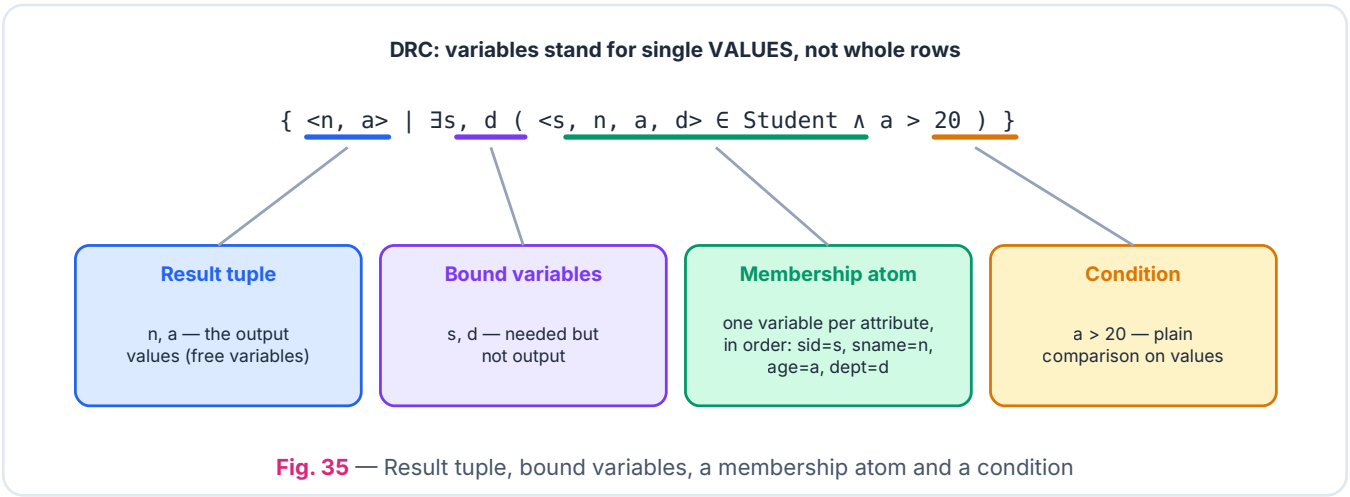


Fig. 35 — Result tuple, bound variables, a membership atom and a condition

Example 1 — selection and projection

Question: names of students older than 20.

DOMAIN CALCULUS

$$\{ \langle n \rangle \mid \exists s, a, d \, (\langle s, n, a, d \rangle \in \text{Student} \wedge a > 20) \}$$

```sql
SELECT sname FROM Student WHERE age > 20;
```

RESULT — 4 ROWS

sname
Ravi
Meena
Sara
Dev

Example 2 — constants inside the tuple

Question: names of CSE students. A constant can sit directly in the atom, which acts as the selection.

DOMAIN CALCULUS

$$\{ \langle n \rangle \mid \exists s, a \, (\langle s, n, a, \text{'CSE'} \rangle \in \text{Student}) \}$$

```sql
SELECT DISTINCT sname FROM Student WHERE dept = 'CSE';
```

RESULT — 3 ROWS

sname
Asha
Meena
Sara

Example 3 — join by reusing a variable

Question: names of students enrolled in C1. The **same variable s** appears in both atoms — that is the join on sid.

DOMAIN CALCULUS

$$\{ \langle n \rangle \mid \exists s, a, d, g \, (\langle s, n, a, d \rangle \in \text{Student} \wedge \langle s, \text{'C1'}, g \rangle \in \text{Enrolled}) \}$$

```sql
SELECT sname FROM Student t
WHERE EXISTS (SELECT 1 FROM Enrolled e
              WHERE e.sid = t.sid AND e.cid = 'C1');
```

RESULT — 4 ROWS

sname
Asha
Ravi
Meena
Sara

Example 4 — "not"

Question: students with no enrolment.

DOMAIN CALCULUS

$$\{ \langle n \rangle \mid \exists s, a, d (\langle s, n, a, d \rangle \in \text{Student} \wedge \neg \exists c, g (\langle s, c, g \rangle \in \text{Enrolled})) \}$$

```sql
SELECT sname FROM Student s
WHERE NOT EXISTS (SELECT 1 FROM Enrolled e WHERE e.sid = s.sid);
```

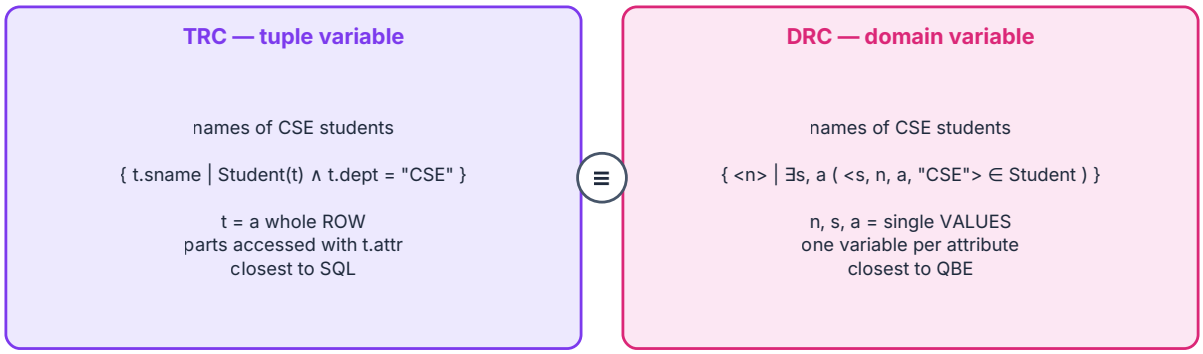
RESULT — 1 ROW

sname
Dev

🧠 **Yaad rakho**

DRC me **har column ka apna variable** hota hai. Jo variable do jagah repeat ho — wahi join hai. Constant seedha tuple ke andar likh sakte ho.

5.2 TRC versus DRC



same query, two notations — both reduce to relational algebra

Fig. 36 — The same query in both notations

	Tuple calculus (TRC)	Domain calculus (DRC)
Variable stands for	A whole tuple	One attribute value
Access an attribute	t.age	the variable itself, e.g. a
Range	Student(t)	<s, n, a, d> ∈ Student
Join	e.sid = t.sid	reuse the same variable
Size of a query	Shorter (one variable per tuple)	Longer (one per attribute)
Influenced	SQL	Query-By-Example (QBE)

5.3 Codd's theorem — algebra equals calculus

📘 **Definition**

Codd's theorem (1972): relational algebra, **safe** tuple relational calculus and **safe** domain relational calculus have exactly the **same expressive power** — every query expressible in one is expressible in the other two.

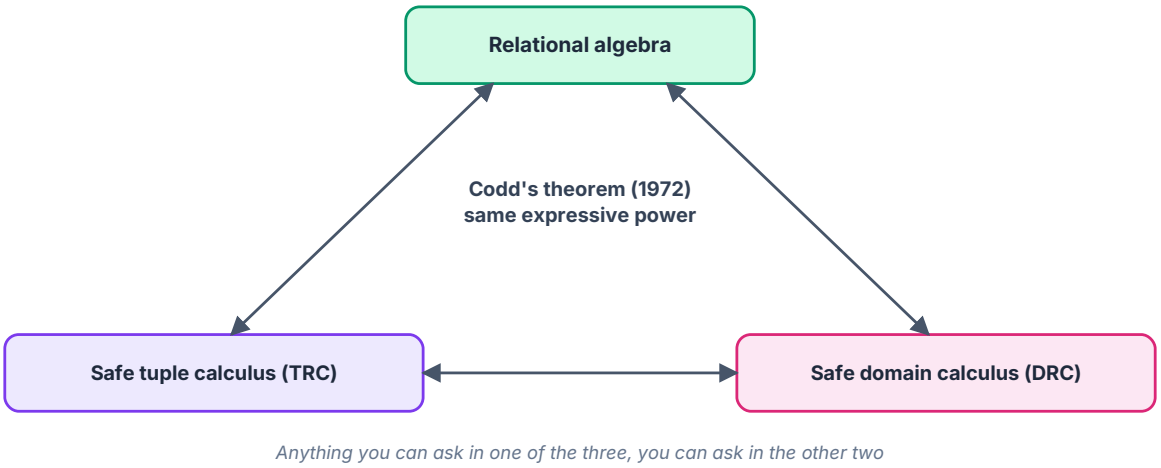


Fig. 37 — The three languages are equivalent — translate freely between them

Algebra operator	What it becomes in safe calculus
$\sigma_\theta(R)$	Range atom plus the condition θ joined with ∧
$\pi_A(R)$	Output only the attributes in A; the others become ∃-bound
R ∪ S · R ∩ S · R − S	∨ · ∧ · ∧ ¬ between the two range atoms
R × S · R ⋈ S	Two range atoms with ∧; the join condition becomes an equality (or a shared variable in DRC)
R ÷ S	∀ … ⇒ ∃ pattern (equivalently ¬∃ … ¬∃)

⭐ **Exam tip — Why safety matters here**

Unrestricted calculus (with unsafe expressions) is **more powerful** than algebra — it can name infinite sets. Restricting to safe expressions removes exactly that excess, which is why the theorem says **safe** calculus.

💡 **Why it matters — Relational completeness**

A query language is **relationally complete** if it can express every query that relational algebra can. SQL is relationally complete (and goes further); that is the formal justification for using it as the standard language.

5.4 What algebra and calculus cannot express

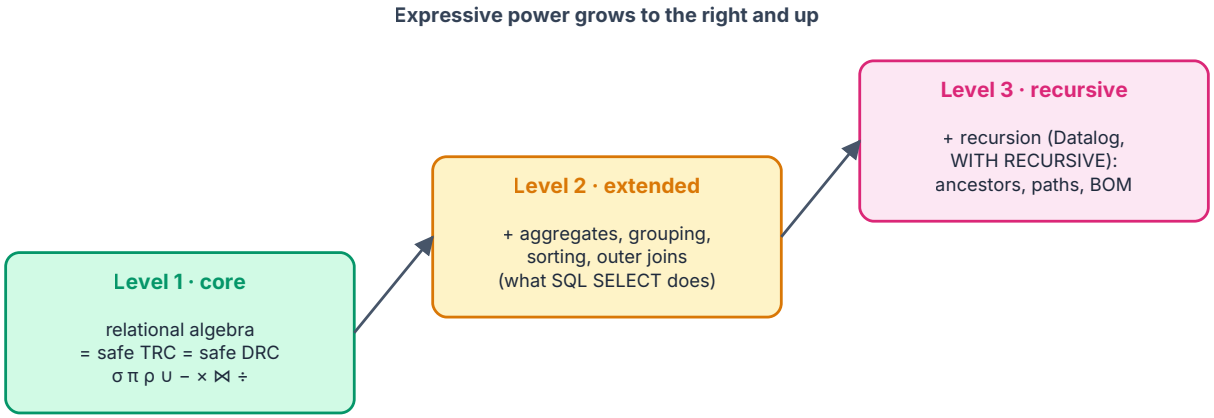


Fig. 38 — Beyond the core you add aggregates, then recursion

- **Aggregates** (count, sum, average) are outside the basic algebra; **extended algebra** adds γ.
- **Ordering** is not part of the theory (relations are unordered); SQL adds `ORDER BY`.
- **Transitive closure / recursion** — "all managers above X at any level", "all reachable cities" — cannot be written with a fixed expression. Recursive SQL (`WITH RECURSIVE`) and Datalog add it.

5.5 Query-By-Example (QBE) in brief

📘 **Definition**

QBE is a graphical language based on domain calculus: you fill in **skeleton tables** instead of writing formulas. **P.** means "print this column", constants and comparisons go in the cell, and an **example element** such as _s that appears in two tables means "the same value" — a join.

Example 1 — select + project: names of CSE students older than 20

Student	sid	sname	age	dept
		P.	> 20	CSE

P. = print this column
condition goes in the cell
blank = don't care

Example 2 — join through a shared example element: names of students in C1

Student	sid	sname	age	dept
	_s	P.		

Enrolled	sid	cid	grade
	_s	C1	

the same example element _s
in both tables = JOIN on sid
(like the same variable in DRC)

Fig. 39 — A selection and a join expressed in QBE

5.6 The master workbook — one question, four languages

Twelve typical questions, each answered in **relational algebra**, **tuple calculus**, **domain calculus** and **SQL**, with the result computed from our sample database. Use this section to **copy the pattern**, then adapt it to your question.

Q1. Names of CSE students

RELATIONAL ALGEBRA

$\pi_{sname}(\sigma_{dept='CSE'}(Student))$

TUPLE CALCULUS

{ t.sname | Student(t) ∧ t.dept = 'CSE' }

DOMAIN CALCULUS

{ <n> | ∃s, a (<s, n, a, 'CSE'> ∈ Student) }

SQL

```
SELECT DISTINCT sname FROM Student WHERE dept = 'CSE';
```

RESULT — 3 ROWS

sname
Asha
Meena
Sara

Q2. Titles of courses worth 4 credits

RELATIONAL ALGEBRA

$\pi_{title}(\sigma_{credits=4}(Course))$

TUPLE CALCULUS

{ c.title | Course(c) ∧ c.credits = 4 }

DOMAIN CALCULUS

{ <t> | ∃i, d (<i, t, 4, d> ∈ Course) }

SQL

```
SELECT title FROM Course WHERE credits = 4;
```

RESULT — 3 ROWS

title
Databases
Signals
AI

Q3. Titles of the courses Ravi is enrolled in

RELATIONAL ALGEBRA

$\pi_{title}(\pi_{sid}(\sigma_{sname='Ravi'}(Student)) \bowtie Enrolled \bowtie Course)$

TUPLE CALCULUS

{ c.title | Course(c) ∧ ∃s ∃e (Student(s) ∧ Enrolled(e) ∧ s.sname = 'Ravi' ∧ e.sid = s.sid ∧ e.cid = c.cid) }

DOMAIN CALCULUS

{ <t> | ∃c, k, d, s, a, p, g (<c, t, k, d> ∈ Course ∧ <s, 'Ravi', a, p> ∈ Student ∧ <s, c, g> ∈ Enrolled) }

```sql
SELECT c.title FROM Student s
JOIN Enrolled e ON s.sid = e.sid JOIN Course c ON c.cid = e.cid
WHERE s.sname = 'Ravi';
```

RESULT — 2 ROWS

title
Databases
Signals

The algebra keeps only *sid* from Student so that the natural join cannot also match on *dept*.

Q4. Students who earned at least one A

RELATIONAL ALGEBRA

$\pi_{sname}(Student \bowtie \sigma_{grade='A'}(Enrolled))$

TUPLE CALCULUS

{ s.sname | Student(s) ∧ ∃e (Enrolled(e) ∧ e.sid = s.sid ∧ e.grade = 'A') }

DOMAIN CALCULUS

{ <n> | ∃s, a, d, c (<s, n, a, d> ∈ Student ∧ <s, c, 'A'> ∈ Enrolled) }

```sql
SELECT DISTINCT s.sname FROM Student s
JOIN Enrolled e ON e.sid = s.sid WHERE e.grade = 'A';
```

RESULT — 3 ROWS

sname
Asha
Meena
Sara

Q5. Students with no enrolment at all

RELATIONAL ALGEBRA

$\pi_{sname}(\text{Student} \triangleright \text{Enrolled}) \equiv \pi_{sname}(\text{Student}) - \pi_{sname}(\text{Student} \bowtie \text{Enrolled})$

TUPLE CALCULUS

$\{ s.sname \mid Student(s) \wedge \neg\exists e\ (Enrolled(e) \wedge e.sid = s.sid) \}$

DOMAIN CALCULUS

$\{ \langle n \rangle \mid \exists s, a, d\ (\langle s, n, a, d \rangle \in Student \wedge \neg\exists c, g\ (\langle s, c, g \rangle \in Enrolled)) \}$

SQL

```
SELECT sname FROM Student s
WHERE NOT EXISTS (SELECT 1 FROM Enrolled e WHERE e.sid = s.sid);
```

RESULT — 1 ROW

sname
Dev

Q6. Students enrolled in both C1 and C2

RELATIONAL ALGEBRA

$\pi_{sname}(\text{Student} \bowtie (\pi_{sid}(\sigma_{cid='C1'}(\text{Enrolled})) \cap \pi_{sid}(\sigma_{cid='C2'}(\text{Enrolled}))))$

TUPLE CALCULUS

$\{ s.sname \mid Student(s) \wedge \exists e1\ \exists e2\ (Enrolled(e1) \wedge Enrolled(e2) \wedge e1.sid = s.sid \wedge e2.sid = s.sid \wedge e1.cid = 'C1' \wedge e2.cid = 'C2') \}$

DOMAIN CALCULUS

$\{ \langle n \rangle \mid \exists s, a, d, g1, g2\ (\langle s, n, a, d \rangle \in Student \wedge \langle s, 'C1', g1 \rangle \in Enrolled \wedge \langle s, 'C2', g2 \rangle \in Enrolled) \}$

SQL

```
SELECT s.sname FROM Student s
WHERE s.sid IN (SELECT sid FROM Enrolled WHERE cid = 'C1'
                INTERSECT
                SELECT sid FROM Enrolled WHERE cid = 'C2');
```

RESULT — 3 ROWS

sname
Asha
Meena
Sara

Q7. Students enrolled in all CSE courses

RELATIONAL ALGEBRA

$\pi_{sname}(Student \bowtie (\pi_{sid, cid}(Enrolled) \div \pi_{cid}(\sigma_{dept='CSE'}(Course))))$

TUPLE CALCULUS

{ s.sname | Student(s) ∧ ∀c ((Course(c) ∧ c.dept = 'CSE') ⇒ ∃e (Enrolled(e) ∧ e.sid = s.sid ∧ e.cid = c.cid)) }

DOMAIN CALCULUS

{ <n> | ∃s, a, d (<s, n, a, d> ∈ Student ∧ ∀c (∃t, k (<c, t, k, 'CSE'> ∈ Course) ⇒ ∃g (<s, c, g> ∈ Enrolled))) }

SQL

```
SELECT s.sid, s.sname FROM Student s
WHERE NOT EXISTS (
  SELECT * FROM Course c WHERE c.dept = 'CSE'
  AND NOT EXISTS (
    SELECT * FROM Enrolled e WHERE e.sid = s.sid AND e.cid = c.cid));
```

RESULT — 2 ROWS

sid	sname
1	Asha
3	Meena

Q8. Students older than Meena

RELATIONAL ALGEBRA

$\pi_{s1.sname}(\sigma_{s1.age > s2.age \wedge s2.sname='Meena'}(\rho_{s1}(Student) \times \rho_{s2}(Student)))$

TUPLE CALCULUS

{ s1.sname | Student(s1) ∧ ∃s2 (Student(s2) ∧ s2.sname = 'Meena' ∧ s1.age > s2.age) }

DOMAIN CALCULUS

{ <n> | ∃s, a, d, s2, a2, d2 (<s, n, a, d> ∈ Student ∧ <s2, 'Meena', a2, d2> ∈ Student ∧ a > a2) }

SQL

```
SELECT s1.sname FROM Student s1 JOIN Student s2
  ON s1.age > s2.age WHERE s2.sname = 'Meena';
```

RESULT — 1 ROW

sname
Dev

Q9. Students who take a course from another department

RELATIONAL ALGEBRA

$$\pi_{sname}(\sigma_{dept \neq cdept}(Student \bowtie Enrolled \bowtie \rho_{dept \to cdept}(Course)))$$

TUPLE CALCULUS

$$\{ s.sname \mid Student(s) \wedge \exists e \exists c (Enrolled(e) \wedge Course(c) \wedge e.sid = s.sid \wedge e.cid = c.cid \wedge s.dept \neq c.dept) \}$$

DOMAIN CALCULUS

$$\{ \langle n \rangle \mid \exists s, a, p, c, g, t, k, d (\langle s, n, a, p \rangle \in Student \wedge \langle s, c, g \rangle \in Enrolled \wedge \langle c, t, k, d \rangle \in Course \wedge p \neq d) \}$$

SQL

```
SELECT DISTINCT s.sname FROM Student s
JOIN Enrolled e ON s.sid = e.sid JOIN Course c ON c.cid = e.cid
WHERE s.dept <> c.dept;
```

RESULT — 1 ROW

sname
Ravi

Here ρ renames Course's *dept* to *cdept* — without it the natural join would force the two departments to be equal.

Q10. Number of students and average age per department

EXTENDED RELATIONAL ALGEBRA

$$\Upsilon_{dept \; ; \; count(*) \to n, \; avg(age) \to avg_age}(Student)$$

SQL

```
SELECT dept, COUNT(*) AS n, AVG(age) AS avg_age
FROM Student GROUP BY dept ORDER BY dept;
```

RESULT — 3 ROWS

dept	n	avg_age
CSE	3	21.0
ECE	2	22.0
ME	1	20.0

Plain TRC and DRC have no aggregates; extended calculus adds count / avg as functions over a set of tuples.

Q11. The oldest student

RELATIONAL ALGEBRA

$\pi_{sname}(Student) - \pi_{s1.sname}(\rho_{s1}(Student) \bowtie_{s1.age < s2.age} \rho_{s2}(Student))$

TUPLE CALCULUS

{ s.sname | Student(s) ∧ ¬∃s2 (Student(s2) ∧ s2.age > s.age) }

DOMAIN CALCULUS

{ <n> | ∃s, a, d (<s, n, a, d> ∈ Student ∧ ¬∃s2, n2, a2, d2 (<s2, n2, a2, d2> ∈ Student ∧ a2 > a)) }

SQL

```
SELECT sname FROM Student
EXCEPT
SELECT s1.sname FROM Student s1 JOIN Student s2 ON s1.age < s2.age;
```

RESULT — 1 ROW

sname
Dev

Q12. Pairs of students from the same department

RELATIONAL ALGEBRA

$\pi_{s1.sname,\ s2.sname}(\rho_{s1}(Student) \bowtie_{s1.dept = s2.dept \wedge s1.sid < s2.sid} \rho_{s2}(Student))$

TUPLE CALCULUS

{ s1.sname, s2.sname | Student(s1) ∧ Student(s2) ∧ s1.dept = s2.dept ∧ s1.sid < s2.sid }

DOMAIN CALCULUS

{ <n1, n2> | ∃i1, a1, d, i2, a2 (<i1, n1, a1, d> ∈ Student ∧ <i2, n2, a2, d> ∈ Student ∧ i1 < i2) }

SQL

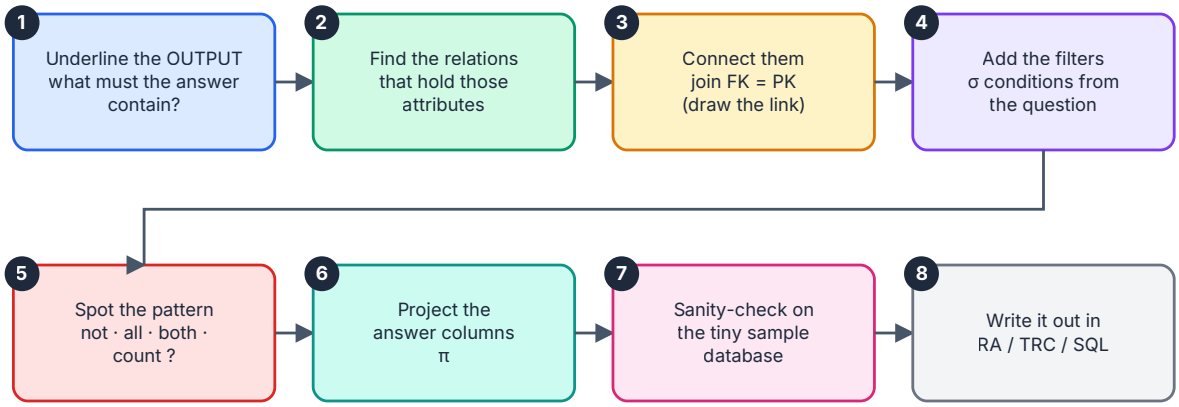
```
SELECT s1.sname AS student1, s2.sname AS student2
FROM Student s1 JOIN Student s2
  ON s1.dept = s2.dept AND s1.sid < s2.sid;
```

RESULT — 4 ROWS

student1	student2
Asha	Meena
Asha	Sara
Ravi	Dev
Meena	Sara

In the domain calculus the shared variable *d* in both atoms expresses "same department".

5.7 A method for writing any query



Do the steps in order — most wrong queries skip step 1 or step 5

Fig. 40 — Eight steps from English to a correct query

Worked example — *"Names of students who take a course from another department."*

1. **Output:** sname.
2. **Relations holding it:** Student (sname, dept) and Course (dept) — and Enrolled to connect them.
3. **Connect:** Student.sid = Enrolled.sid, Enrolled.cid = Course.cid.
4. **Filter:** Student.dept ≠ Course.dept (both columns are called *dept* — rename one!).
5. **Pattern?** "another department" is a plain comparison; no not / all / both.
6. **Project** sname.
7. **Sanity-check:** Ravi (ECE) takes C1 (CSE) — he must appear; Asha (CSE) takes only CSE courses — she must not. ✔

5.8 Common mistakes and how to avoid them

#	Mistake	Fix
1	Natural join on an accidental common attribute (e.g. *dept*)	Project the column away first, rename it with ρ, or use an explicit `JOIN … ON`
2	$\sigma_{cid \neq 'C2'}$ for "not enrolled in C2"	Subtract: everyone − those who do take C2
3	π before σ drops the column the condition needs	Select first, then project
4	Forgetting `DISTINCT` when turning π into SQL	Add `DISTINCT` (algebra works on sets)
5	Union / difference of relations with different columns	Check degree and domains first
6	Writing `= NULL`; forgetting three-valued logic	Use `IS NULL`; remember UNKNOWN is not TRUE
7	Wrong divisor or direction in ÷	A(x, y) ÷ B(y): the divisor's attributes must be a subset of the dividend's
8	`NOT IN` over a column that may contain NULL	Use `NOT EXISTS`
9	∀ without an ∃ (vacuous truth lets empty cases through)	Add an existence test when empties should not qualify
10	An unsafe calculus expression such as { t | ¬R(t) }	Restrict every variable with a range atom
11	Self-join without renaming the two copies	Use ρ_{s1}, ρ_{s2} and qualify every attribute

5.9 Practice problems with solutions

Try each problem first, then compare. The result shown is computed from the sample database.

P1. Names and ages of ECE students

ANSWER

$$\pi_{sname,\ age}(\sigma_{dept='ECE'}(Student))$$

```sql
SELECT sname, age FROM Student WHERE dept = 'ECE';
```

RESULT — 2 ROWS

sname	age
Ravi	21
Dev	23

P2. Titles of CSE courses worth more than 3 credits

ANSWER

$\pi_{title}(\sigma_{dept='CSE' \wedge credits>3}(Course))$

```sql
SELECT title FROM Course WHERE dept = 'CSE' AND credits > 3;
```

RESULT — 2 ROWS

title
Databases
AI

P3. Ids of students who earned an A in some course

ANSWER

$\pi_{sid}(\sigma_{grade='A'}(Enrolled))$

```sql
SELECT DISTINCT sid FROM Enrolled WHERE grade = 'A';
```

RESULT — 3 ROWS

sid
1
3
5

P4. Names of students enrolled in *Networks*

ANSWER

$\pi_{sname}(\pi_{sid,\ sname}(Student) \bowtie Enrolled \bowtie \pi_{cid}(\sigma_{title='Networks'}(Course)))$

```sql
SELECT DISTINCT s.sname FROM Student s
JOIN Enrolled e ON s.sid = e.sid JOIN Course c ON c.cid = e.cid
WHERE c.title = 'Networks';
```

RESULT — 3 ROWS

sname
Asha
Meena
Sara

P5. Titles of courses that no ME student takes

ANSWER

$$\pi_{title}(\text{Course} \bowtie (\pi_{cid}(\text{Course}) - \pi_{cid}(\text{Enrolled} \bowtie \pi_{sid}(\sigma_{dept='ME'}(\text{Student})))))$$

```sql
SELECT title FROM Course
WHERE cid NOT IN (SELECT e.cid FROM Enrolled e
                  JOIN Student s ON s.sid = e.sid WHERE s.dept = 'ME');
```

RESULT — 4 ROWS

title
Databases
Networks
Signals
AI

P6. Pairs of students with the same age

ANSWER

$$\pi_{s1.sname,\ s2.sname}(\rho_{s1}(\text{Student}) \bowtie_{s1.age = s2.age \wedge s1.sid < s2.sid} \rho_{s2}(\text{Student}))$$

```sql
SELECT s1.sname, s2.sname AS sname2 FROM Student s1 JOIN Student s2
  ON s1.age = s2.age AND s1.sid < s2.sid;
```

RESULT — 2 ROWS

sname	sname2
Asha	Karan
Ravi	Sara

P7. Students enrolled in at least two courses

ANSWER

$$\pi_{sname}(\sigma_{n \geq 2}(\gamma_{sid,\ sname\ ;\ count(*) \rightarrow n}(\pi_{sid,\ sname}(\text{Student}) \bowtie \text{Enrolled})))$$

```sql
SELECT s.sname FROM Student s JOIN Enrolled e ON s.sid = e.sid
GROUP BY s.sid, s.sname HAVING COUNT(*) >= 2;
```

RESULT — 4 ROWS

sname
Asha
Ravi
Meena
Sara

P8. Students enrolled in all ECE courses

ANSWER

$$\pi_{sname}(Student \bowtie (\pi_{sid,\ cid}(Enrolled) \div \pi_{cid}(\sigma_{dept='ECE'}(Course))))$$

```sql
SELECT s.sname FROM Student s
WHERE NOT EXISTS (SELECT * FROM Course c WHERE c.dept = 'ECE'
  AND NOT EXISTS (SELECT * FROM Enrolled e
                  WHERE e.sid = s.sid AND e.cid = c.cid));
```

RESULT — 1 ROW

sname
Ravi

P9. Departments with at least two students, and their average age

ANSWER

$$\pi_{dept,\ avg_age}(\sigma_{n \geq 2}(\gamma_{dept\ ;\ count(*) \rightarrow n,\ avg(age) \rightarrow avg_age}(Student)))$$

```sql
SELECT dept, AVG(age) AS avg_age FROM Student
GROUP BY dept HAVING COUNT(*) >= 2 ORDER BY dept;
```

RESULT — 2 ROWS

dept	avg_age
CSE	21.0
ECE	22.0

P10. Students who did not earn an A in any course (including those with no enrolment)

ANSWER

$$\pi_{sname}(Student) - \pi_{sname}(Student \bowtie \sigma_{grade='A'}(Enrolled))$$

```sql
SELECT sname FROM Student s
WHERE NOT EXISTS (SELECT 1 FROM Enrolled e WHERE e.sid = s.sid AND e.grade = 'A');
```

RESULT — 3 ROWS

sname
Ravi
Karan
Dev

5.10 Quick viva answers

Question	One-line answer
What is relational algebra?	A procedural query language of operators that take and return relations
Name the six basic operators	σ, π, ρ, ∪, −, ×
Which operators are derived?	∩, ⋈, ÷ (and ⋉, ▷)
σ versus π?	σ chooses rows; π chooses columns and removes duplicates
Why are duplicates removed?	A relation is a set
What is union-compatibility?	Same number of attributes with matching domains
Natural join versus equi-join?	Natural joins on all same-named attributes and drops the duplicate column
What does an outer join add?	Unmatched tuples, padded with NULL
When do you use division?	For "for all / every" queries
Why do we need ρ?	Self-joins and attribute-name clashes
TRC versus DRC?	Variable = whole tuple / variable = single value
Free versus bound variable?	Free = appears in the answer; bound = captured by ∃ or ∀
What is a safe expression?	One that always returns a finite set of values from the database
State Codd's theorem	Algebra = safe TRC = safe DRC in expressive power
What does "relationally complete" mean?	At least as expressive as relational algebra
Can basic algebra compute an average?	No — extended algebra adds γ
Can algebra express transitive closure?	No — it needs recursion
Set or bag — algebra versus SQL?	Algebra uses sets; SQL uses bags unless you write DISTINCT

❓ Likely exam questions — Unit V

- Explain domain relational calculus with an example.
- Compare TRC and DRC.
- State and explain Codd's theorem; why must expressions be safe?
- What is relational completeness? Is SQL relationally complete?
- What can relational algebra not express? Give examples.
- Write a query in algebra, TRC, DRC and SQL (students in all CSE courses).
- Explain QBE with an example.
- Describe the steps you follow to write a complex query.

Quick comparison sheet

One-glance tables for the last day before the exam.

Operators at a glance

Operator	Symbol	Result	Remember
Selection	σ	Subset of rows, same columns	Filters with a condition · SQL `WHERE`
Projection	π	Chosen columns, duplicates removed	Pick columns · SQL `SELECT DISTINCT`
Rename	ρ	Same data, new names	Essential for self-joins
Union / Intersection / Difference	∪ ∩ −	Rows by set rule	Need union-compatibility · difference is not commutative
Product	×	m × n rows	σ over × = join
Theta / equi / natural join	⋈	Matching pairs only	Natural = same-named attributes, beware accidental matches
Outer joins	$⋈^{L\ R}_{F}$	Inner join + unmatched, NULL-padded	Left / right / full
Semi / anti-join	⋉ ▷	Rows of R with / without a match	SQL `EXISTS` / `NOT EXISTS`
Division	÷	x-values paired with **all** y in B	"for all" · A(x,y) ÷ B(y)
Grouping / aggregation	γ	One row per group	HAVING = σ after γ

Calculus at a glance

Idea	TRC	DRC
Query form	{ t | P(t) }	{ <x1,…,xn> | P }
Range	Student(t)	<s,n,a,d> ∈ Student
Join	e.sid = t.sid	shared variable
"not"	¬∃e (…)	¬∃c,g (…)
"all"	∀c (P(c) ⇒ ∃e (…))	same shape with value variables
Safety	Every variable must be tied to a stored relation so the answer is finite	
Key theorem	Algebra ≡ safe TRC ≡ safe DRC (Codd, 1972)	

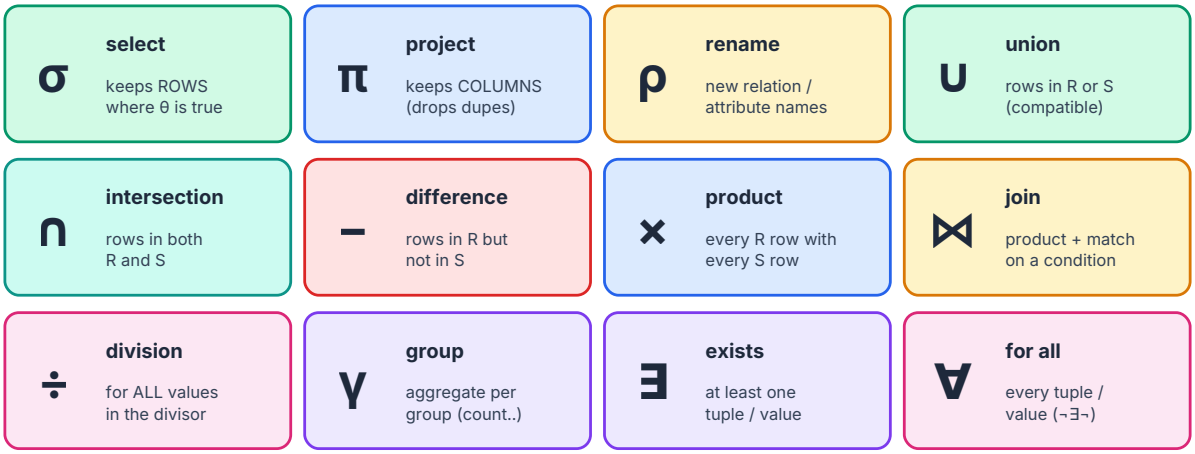


Fig. 41 — The twelve symbols you must be able to read and write

Key facts

Topic	Remember
Relation	A set of tuples — no duplicates, no order, atomic values
Keys	Superkey ⊃ candidate (minimal) ⊃ primary; foreign key refers to a primary key
Six basic operators	σ π ρ ∪ − ×
Closure	Every operator returns a relation, so expressions nest
Union-compatible	Same degree and domains
Division formula	$\pi_x(A) - \pi_x(\,(\,\pi_x(A) \times B\,) - A\,)$
Optimization	Push σ and π down; turn σ over × into ⋈; pick join order
Quantifier duality	$\forall x\ P \equiv \neg\exists x\ \neg P \cdot P \Rightarrow Q \equiv \neg P \vee Q$
Beyond the core	Aggregates (γ) and recursion are extensions

Exam-day revision checklist

Tick these off — if you can **write or draw** each one from memory, you are well covered.

- [ ] Relation terms: tuple, attribute, domain, degree, cardinality
- [ ] Schema vs instance
- [ ] Superkey, candidate, primary and foreign key
- [ ] The four integrity constraints
- [ ] Closure property; set vs bag semantics
- [ ] σ and π with examples
- [ ] Why order matters: σ before π
- [ ] Rename ρ and when it is essential
- [ ] Union-compatibility conditions
- [ ] ∪ ∩ − with Venn diagrams; R ∩ S = R − (R − S)
- [ ] Cartesian product and its row count
- [ ] Product followed by σ = join
- [ ] Theta vs equi vs natural join
- [ ] The natural-join trap (accidental attributes)
- [ ] Outer joins: left, right, full
- [ ] Semi-join and anti-join with SQL forms
- [ ] "Not" queries by difference, not by ≠
- [ ] Self-join with ρ
- [ ] Division and its π, ×, − derivation
- [ ] Grouping γ and HAVING as σ after γ
- [ ] Equivalence rules; push selection down
- [ ] TRC form, range, free and bound variables
- [ ] ∃, ∀ and their De Morgan duality
- [ ] ¬∃ for "not"; ∀ … ⇒ ∃ for "all"
- [ ] Vacuous truth of ∀
- [ ] Safe vs unsafe expressions
- [ ] DRC form, membership atom, shared variables
- [ ] TRC vs DRC comparison
- [ ] Codd's theorem and relational completeness
- [ ] What algebra cannot express; QBE basics
- [ ] Write one query in algebra, TRC, DRC and SQL

Recommended resources

Resource	Covers
Silberschatz, Korth & Sudarshan — *Database System Concepts*	Relational model, algebra, TRC and DRC (standard textbook)
Elmasri & Navathe — *Fundamentals of Database Systems*	Algebra, calculus, division, outer joins
Ramakrishnan & Gehrke — *Database Management Systems*	Algebra, calculus, QBE, query optimization
Garcia-Molina, Ullman & Widom — *Database Systems: The Complete Book*	Bag vs set algebra, extended operators
E. F. Codd, "A Relational Model of Data for Large Shared Data Banks" (1970); "Relational Completeness of Data Base Sublanguages" (1972)	The original papers behind the model and Codd's theorem
RelaX — relational algebra calculator (dbis-uibk.github.io/relax)	Run algebra expressions and see the result
SQLite, MySQL, PostgreSQL or Oracle	Run every SQL example in this book on the sample database
geeksforgeeks.org — DBMS articles	Extra practice questions

All diagrams in this book were drawn for this book. Every result table was produced by running the shown SQL on the sample database (SQLite). Per-topic "GFG" links open GeeksforGeeks **search** pages. Check university-specific slides for local notation, for example the symbol used for outer joins.